The Peer-to-Peer Paradigm

Slides realized with the collaboration of:

Alberto Montresor
Department of Computer Science, University of Bologna

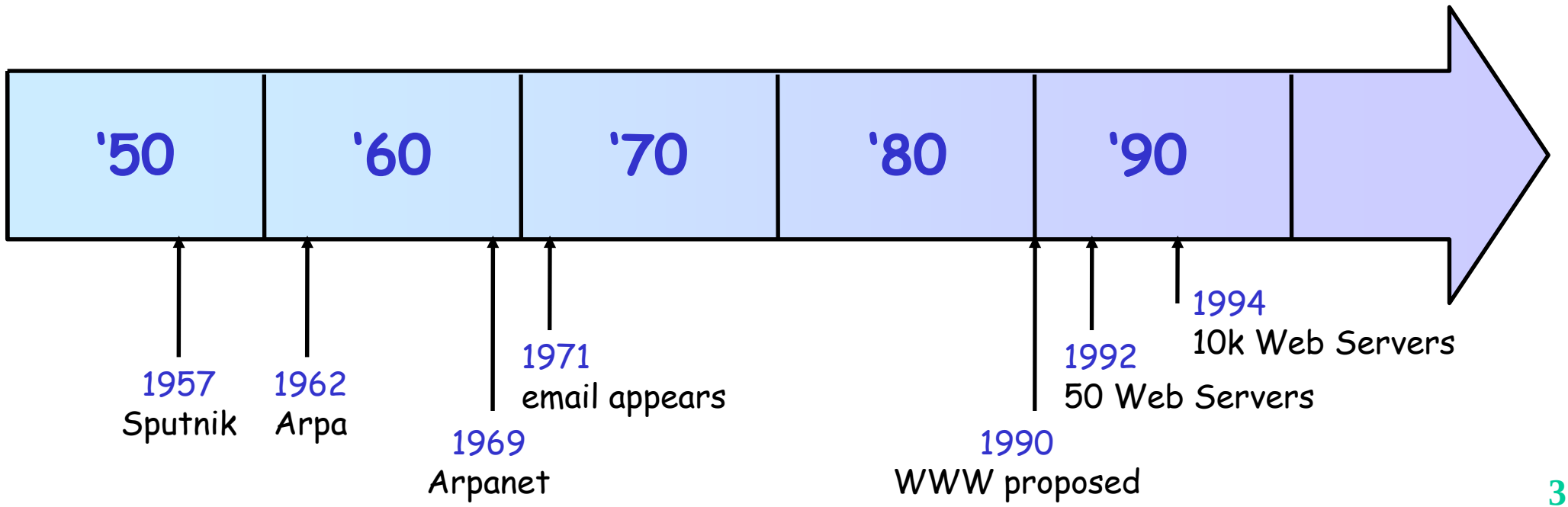
Introduction

- **Recently, the *peer-to-peer* (P2P) paradigm has gained attention from both industry and the media**
- **Peer-to-peer: “basic” definition**
  - A P2P system is a distributed collection of ***peer*** nodes
  - ***Each node is both a server and a client***:
    - may provide services to other peers
    - may consume services from other peers
- **Completely different from the client-server model**

- **Why P2P in a course about Complex Adaptive Systems?**

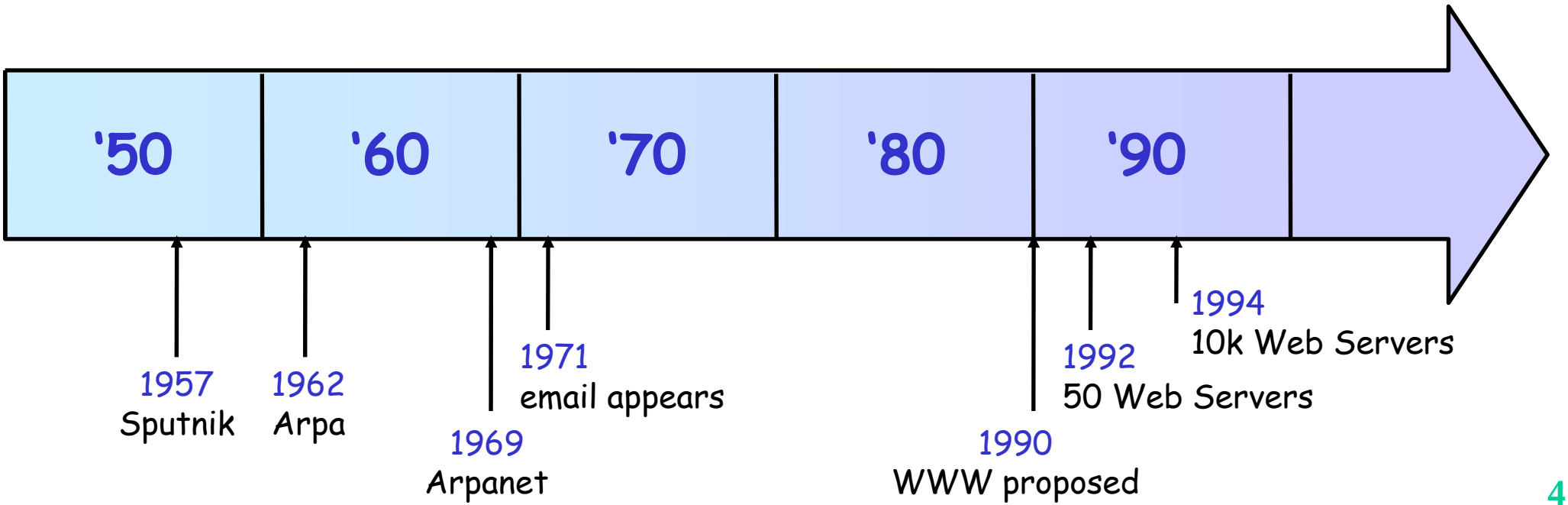
P2P History: 1969 - 1990

- **1969 – 1990: the origins**
  - In the beginnings, all nodes in Arpanet/Internet were peers
  - Every node was capable to
    - perform routing (locate machines)
    - accept ftp connections (file sharing)
    - accept telnet connections (distributed computation)



P2P History: 1995 - 1999

- **1990 – 1999: the Internet explosion**
  - The original “state of grace” was lost
  - Current Internet is organized hierarchically (client/server)
    - Relatively few servers provide services
    - *Client machines are second-class Internet citizens (cut off from the DNS system, dynamic address)*

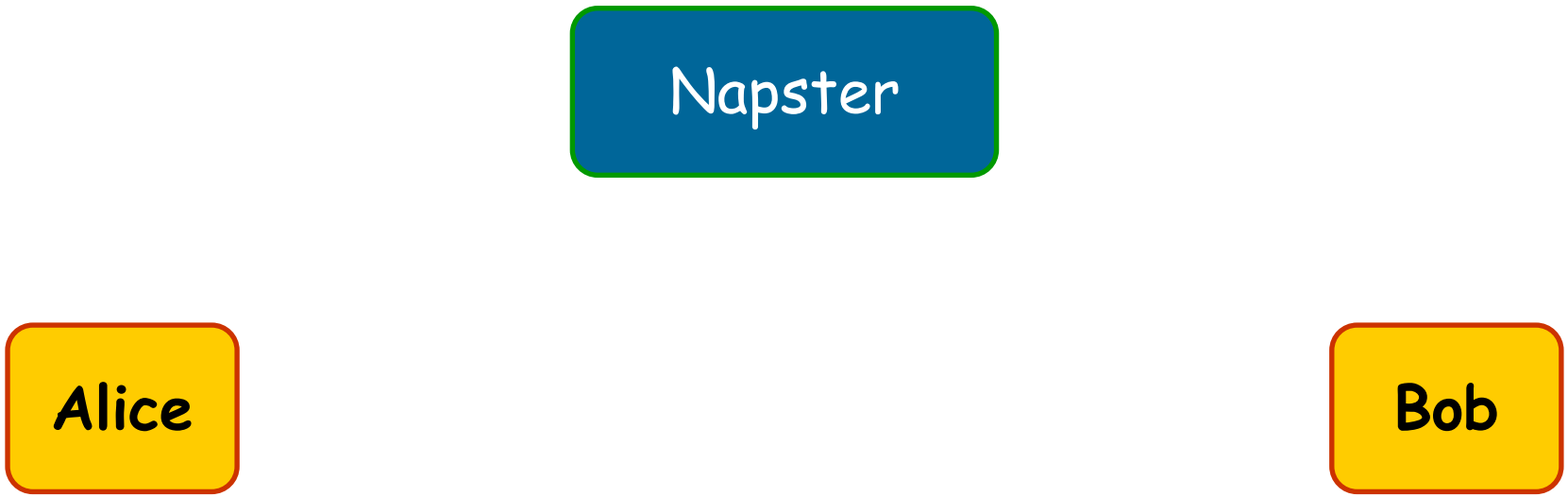


P2P History: 1999 - today

- **1999 – 2001: The advent of Napster**
  - Jan 1999: the first version of *Napster* is released by Shawn Fanning, student at Northeastern University
  - Jul 1999: Napster, Inc. founded
  - Feb 2001: Napster closed down
- **Napster: an enormous success**
  - Jan 2000: Napster unique users > 1.000.000
  - Nov 2000: Napster unique users > 23.000.000
  - Feb 2001: Napster unique users > 50.000.000

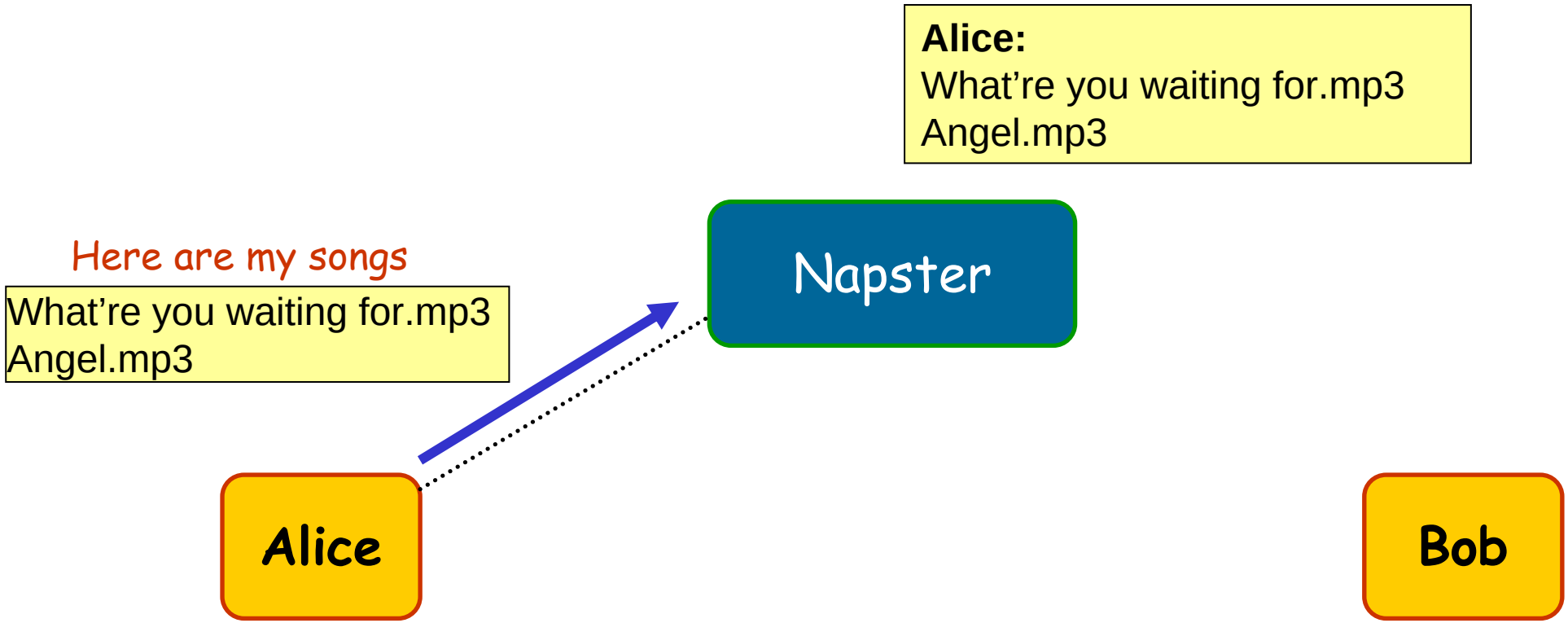
Napster 1/2

- **The Napster architecture:**
  - Client-server w.r.t. searches
  - Peer-to-peer w.r.t downloads



Napster 1/2

- **The Napster architecture:**
  - Client-server w.r.t. searches
  - Peer-to-peer w.r.t downloads

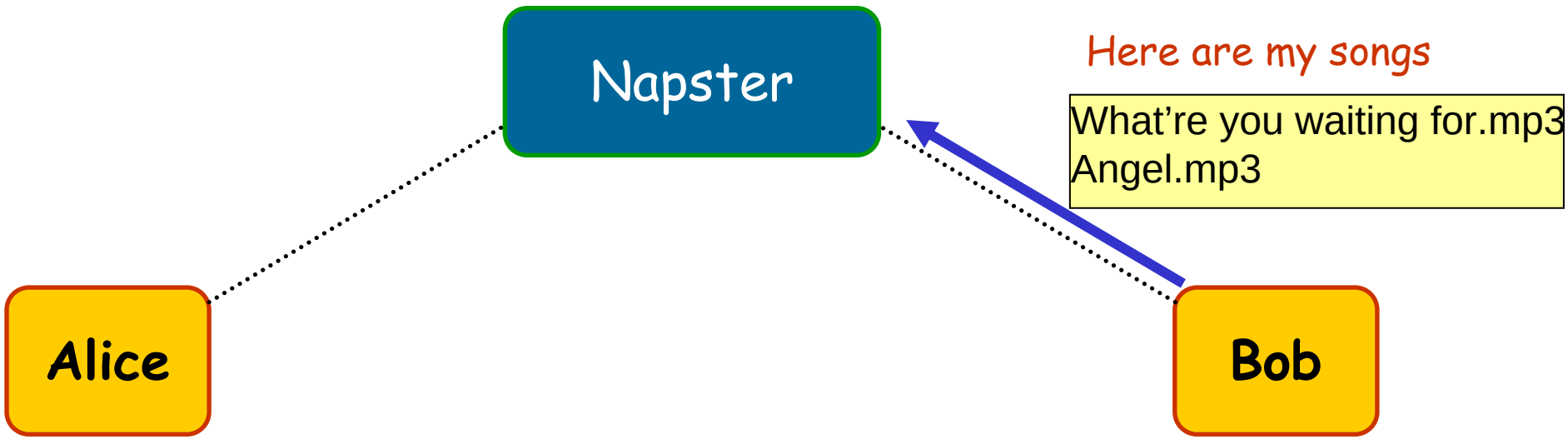


Napster 1/2

- **The Napster architecture:**
  - Client-server w.r.t. searches
  - Peer-to-peer w.r.t downloads

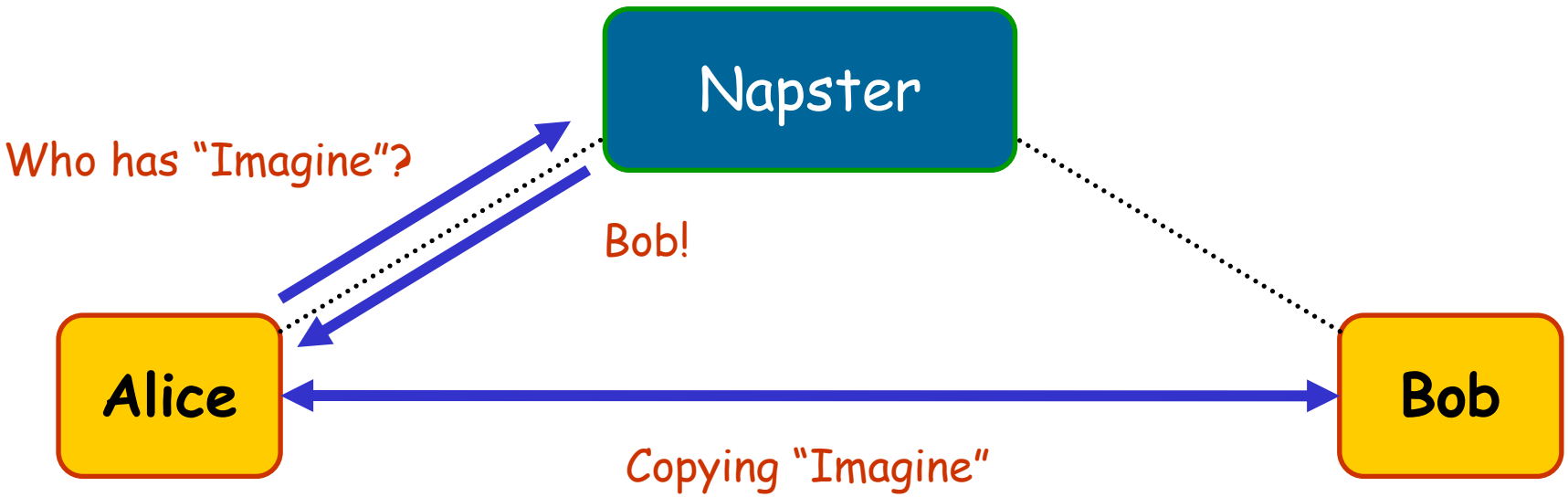
Alice:
What're you waiting for.mp3
Angel.mp3

Bob:
Imagine.mp3
Angel.mp3



Napster 2/2

- **The Napster architecture:**
  - Client-server w.r.t. searches
  - Peer-to-peer w.r.t downloads



Alternative Definitions of Peer-to-Peer

- **Napster is not "pure P2P"**
- **How to characterize the peer-to-peer paradigm?**
- **Peer-to-peer applications**
  - Put together resources available at machines located *at the edges of the Internet*
  - Share computer resources and services by *direct exchange* between peer nodes
  - Employ distributed resources to perform a critical *function* in a *decentralized* manner

Why P2P?

- **P2P is extremely interesting from a technical point of view**
- **Its completely decentralized model enables the development of applications with**
  - *high-availability*
  - *fault-tolerance*
  - *scalability*

  characteristics previously unseen in Internet
- **It exploits what has been defined the “dark matter” of Internet**

Levels of P2P-ness

- **P2P as a mindset**
  - Slashdot
- **P2P as a model**
  - Gnutella
- **P2P as an implementation choice**
  - Application-layer multicast
- **P2P as an inherent property**
  - Ad-hoc networks

Areas of applicability of P2P

- sharing of *content*
  - file sharing, content delivery network
  - Gnutella, Kazaa, Akamai
  - CareScience Care Data Exchange
    - exchange of information between healthcare organizations:
      - » clinical results
      - » patient demographics
      - » medical records
    - Aimed at giving medical personnel easy access to crucial health-related data
    - In the United States, 50.000 deaths per year are caused by the lack of information
- sharing of *storage*
  - distributed file system, distributed search engine
  - OceanStore
- sharing of *CPU time*
  - parallel computing
  - Grid, Seti@Home
- sharing of *communication channels*
  - publish-subscribe

P2P Topologies

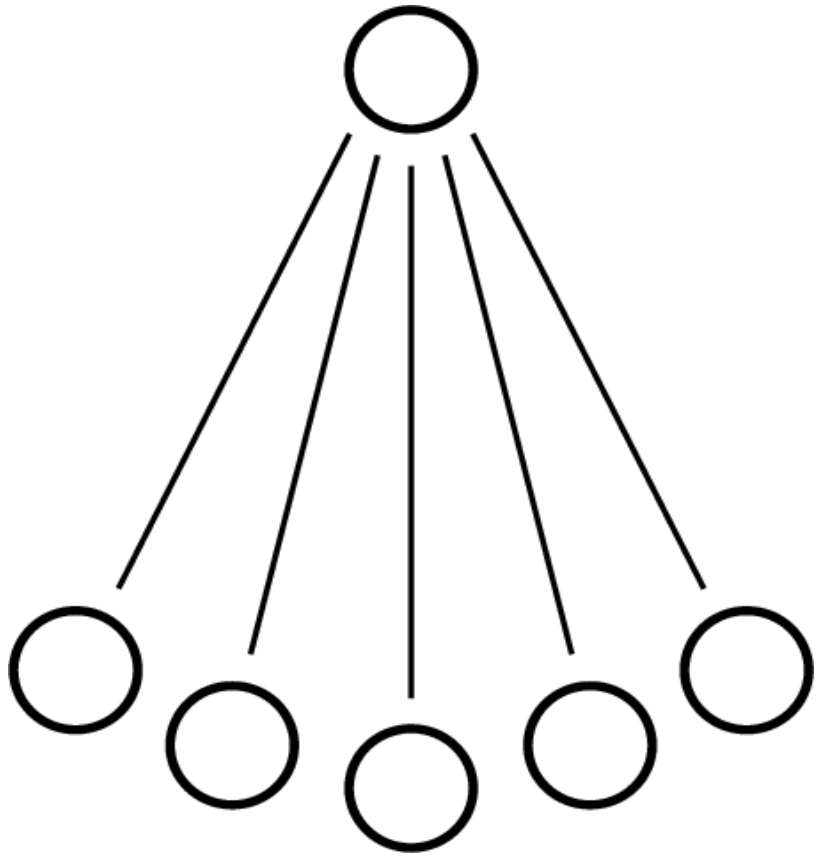
- **Centralized**
- **Hierarchical**
- **Decentralized**
- **Hybrid**

Evaluating topologies

- **Manageability**
  - How hard is it to keep working?
- **Information coherence**
  - How authoritative is info?
- **Extensibility**
  - How easy is it to grow?
- **Fault tolerance**
  - How well can it handle failures?
- **Resistance to legal or political intervention**
  - How hard is it to shut down? (good or bad)
- **Security**
  - How hard is it to subvert?
- **Scalability**
  - How big can it grow?

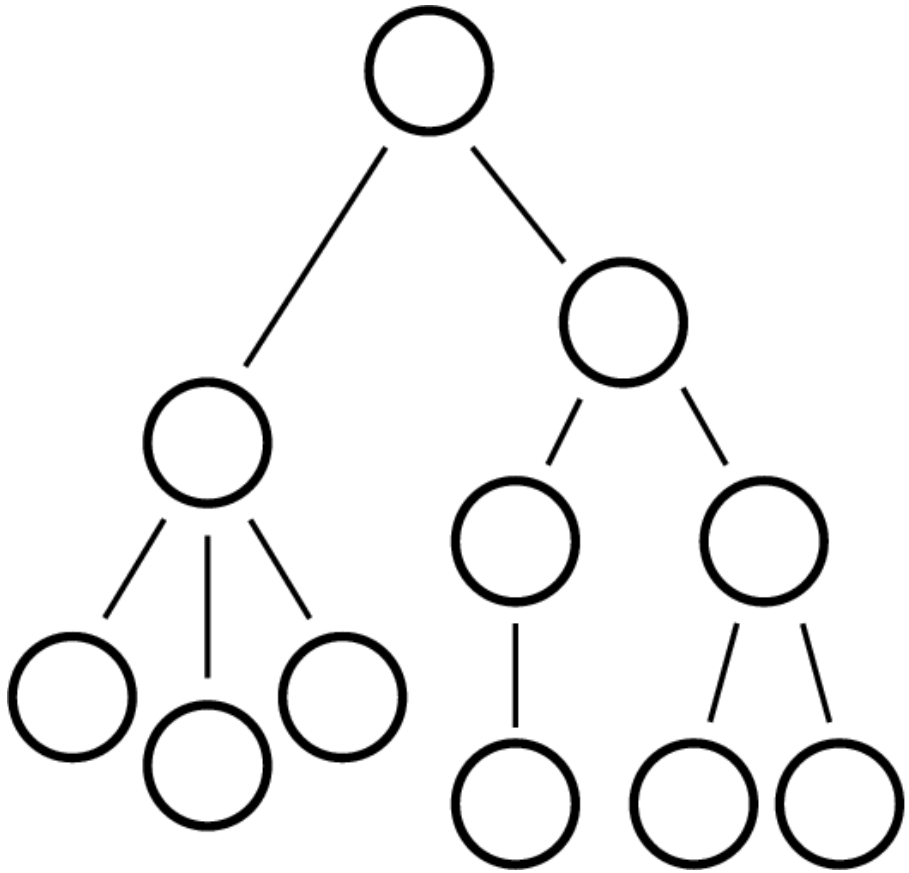
Centralized

Manageable	✓	**System is all in one place**
Coherent	✓	**Information is centralized**
Extensible	X	**No**
Fault Tolerant	X	**Single point of failure**
Secure	✓	**Simply secure one host**
Lawsuit-proof	X	**Easy to shut down**
Scalable	?	**In theory, no**



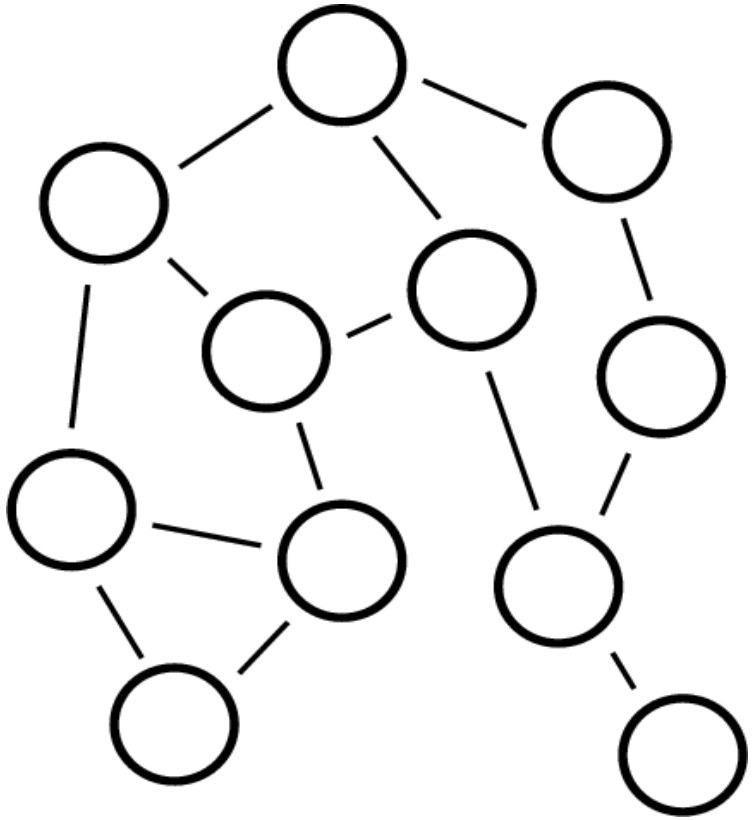
Hierarchical

Manageable	½	**Chain of authority**
Coherent	½	**Cache consistency**
Extensible	½	**Add more leaves, rebalance**
Fault Tolerant	½	**Root is vulnerable**
Secure	X	**Too easy to spoof links**
Lawsuit-proof	X	**Just shut down the root**
Scalable	✓	**Hugely scalable – DNS**



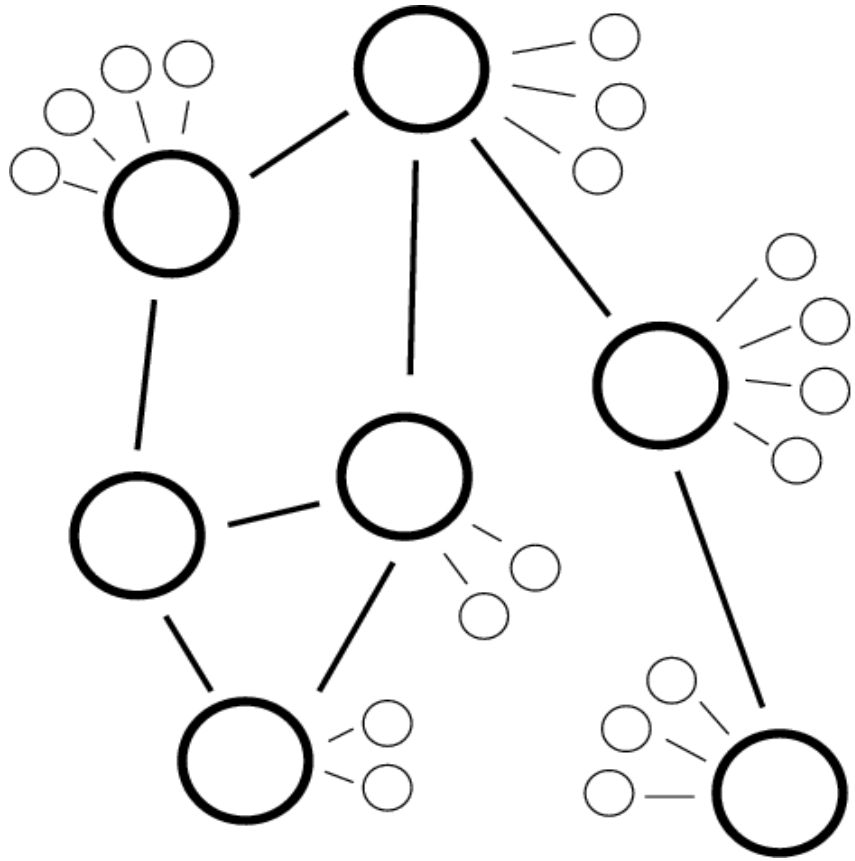
Decentralized

Manageable	X	**Very difficult, many owners**
Coherent	X	**Difficult, unreliable peers**
Extensible	✓	**Anyone can join in!**
Fault Tolerant	✓	**Redundancy**
Secure	X	**Difficult, open research**
Lawsuit-proof	✓	**No one to sue**
Scalable	?	**Theory – yes : Practice – no**



Centralized + Decentralized (Super-Peer)

Manageable	X	**Same as decentralized**
Coherent	½	**Better than decentralized**
Extensible	✓	**Anyone can still join!**
Fault Tolerant	✓	**Plenty of redundancy**
Secure	X	**Same as decentralized**
Lawsuit-proof	✓	**Still no one to sue**
Scalable	?	**Looking very promising**



Best architecture for P2P networks?

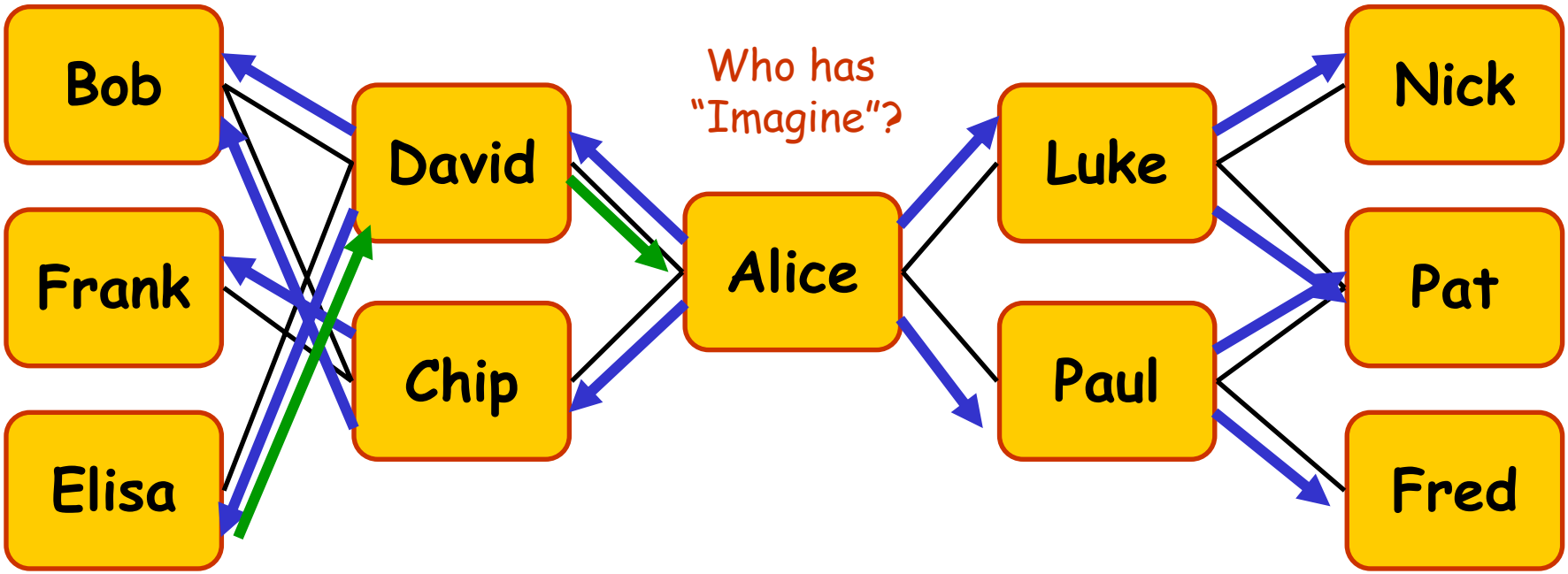
Evolution of P2P models

- **Unstructured, flooding-based**
  - **Gnutella**
- **Unstructured, document-routing**
  - **Freenet**
- **Structured, document-routing**
  - Distributed Hash Tables
  - **CAN**, Chord, Pastry, Tapestry
- **Hybrid, document routing**
  - **BitTorrent**

Gnutella

- **The Gnutella protocol consists of:**
  - a set of message types
    - representing the ways in which peers (i.e. servents) communicate over the network
  - a set of rules
    - governing the inter-servent exchange of messages
- **How to connect to a Gnutella network**
  - A Gnutella servent connects to the network by establishing a connection with another servent currently on the network
  - The acquisition of another servent's address is not part of the protocol definition
    - "Out-of-band" methods

Two types of messages



Two types of messages

- **Broadcast**
  - sent to all nodes with which the sender has open TCP connections
  - This poses serious problems of scalability
  - Mechanisms are used to reduce the number of messages
- **Back-propagate**
  - sent on a specific connection on the reverse of the path taken by an initial broadcasted message

Gnutella Messages

- **Each message is composed of:**
  - A message type field
    - PING, PONG
    - QUERY, QUERYHIT
    - PUSH
  - A Time-To-Live (TTL) Field
  - A 16-byte ID field uniquely identifying the message
    - randomly generated
    - not related to the address of the requester (anonymity)

Gnutella Message Types

- **PING/PONG**
  - Used to maintain information about the nodes currently in the network
  - Originally, an "who's there" flooding message
  - A servent receiving a PING is expected to respond with a PONG message
  - A PONG message has the same ID of the corresponding PING message
  - contains:
    - address of connected Gnutella servent
    - total size and total number of files shared by this servent

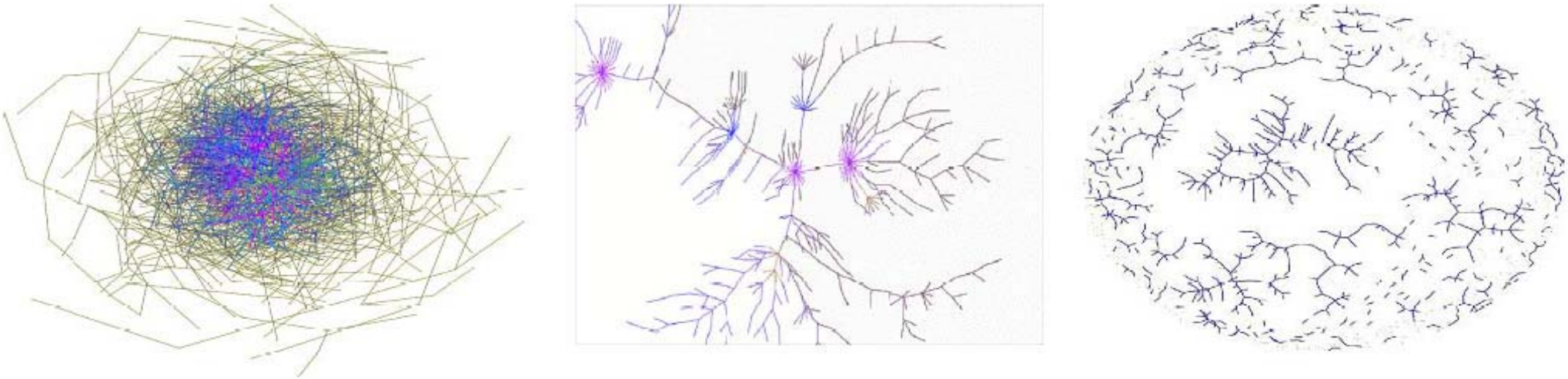
Gnutella Message Types

- **QUERY**
  - The primary mechanism for searching the distributed network
  - Contains the query string
  - A servent is expected to respond with a QUERYHIT message if a match is found against its local data set
- **QUERYHIT**
  - the response to a query
  - has the same ID of the corresponding QUERY message
  - contains enough information to acquire the data matching the corresponding query
    - IP Address + port number
    - List of file names

Gnutella Message Forwarding Rules

- **A servent receiving a message must forward it to each of the other servents to which it is connected**
- **Exceptions:**
  - TTL is zero
  - PONG messages must be routed along the same path of the incoming PING
  - QUERYHIT messages must be routed along the same path of incoming QUERY messages
  - Messages with duplicated ID should be discarded
- **A cache of message IDs, along with sender, is needed**

Network Topology



(a) (b) (c)

Figure 15. Left: Topology of the Gnutella network as of February 16, 2001 (1771 peers); Middle: Topology of the Gnutella network after a random 30% of the nodes are removed; Right: Topology of the Gnutella network after the highest-degree 4% of the nodes are removed.

Small world, scale-free……

User Traffic (Numeric Example)

- **Why Gnutella Can't Scale. By Jordan Rotter**
- **Consider a query string of 18 bytes**
  - IP header 20 bytes
  - TCP header 20 bytes
  - Gnutella header 24 bytes
  - Search string 18 bytes + 1 byte(null)
  - Total: 83 bytes

User Traffic Numeric Example Continued

- **Consider a query string of 18 bytes**
  - With data packet (s) = 83 bytes
  - TTL = 8,
  - Number of connections (n)= 8
  - Number of user reached: $\sum_{i=1}^{TTL} n(n-1)^{(i-1)}$
  - Bandwidth incurred = 1,275,942,400 bytes
  - Therefore, 18 bytes of query generated 1.2GB of traffic!

Free Riding

- **Besides technical problems, non-technical ones such as “free-riding” challenge Gnutella.**
- **“Free-riding” means that most of the Gnutella usres do not provide files to share, they just download**
- **A study conducted on 33335 hosts shows:**
  - 22084 (66%) peers share no files
  - 24347 (73%) peers share less than ten files
  - Top 1% of hosts shares the 37% of all the shared files
  - Top 10% of hosts shares the 87% of all the shared files
  - Top 1% provides 47% of all the answers
  - Top 25% provides 98% of all the answers
- **Almost client-server!**

Gnutella: conclusions

- **Problems**
  - Scalability
  - Amount of traffic generated by Ping/Pong messages
    - caches
    - ultra-peer
  - What kind of topology is generated?
    - Is it planned?
    - Is it good?

Freenet

- **Freenet:**
  - An adaptive peer-to-peer application that permits the publication, replication and retrieval of data while protecting the anonymity of users
- **Philosophy:**

I worry about my child and the Internet all the time, even though she's too young to have logged on yet. Here's what I worry about. I worry that 10 or 15 years from now, she will come to me and say 'Daddy, where were you when they took freedom of the press away from the Internet?'"

Mike Godwin, Electronic Frontier Foundation

Freenet Goals

- **Socio-political goals of Freenet**
  - Publisher anonymity
  - Reader anonymity
  - Server anonymity
  - Resistance to attempts by attackers to deny access to data
- **Technical goals of Freenet**
  - Decentralization of all network functions
  - Data replication/distribution without human intervention
  - Efficient dynamic storage and routing of information
  - High availability for popular data

Freenet Design

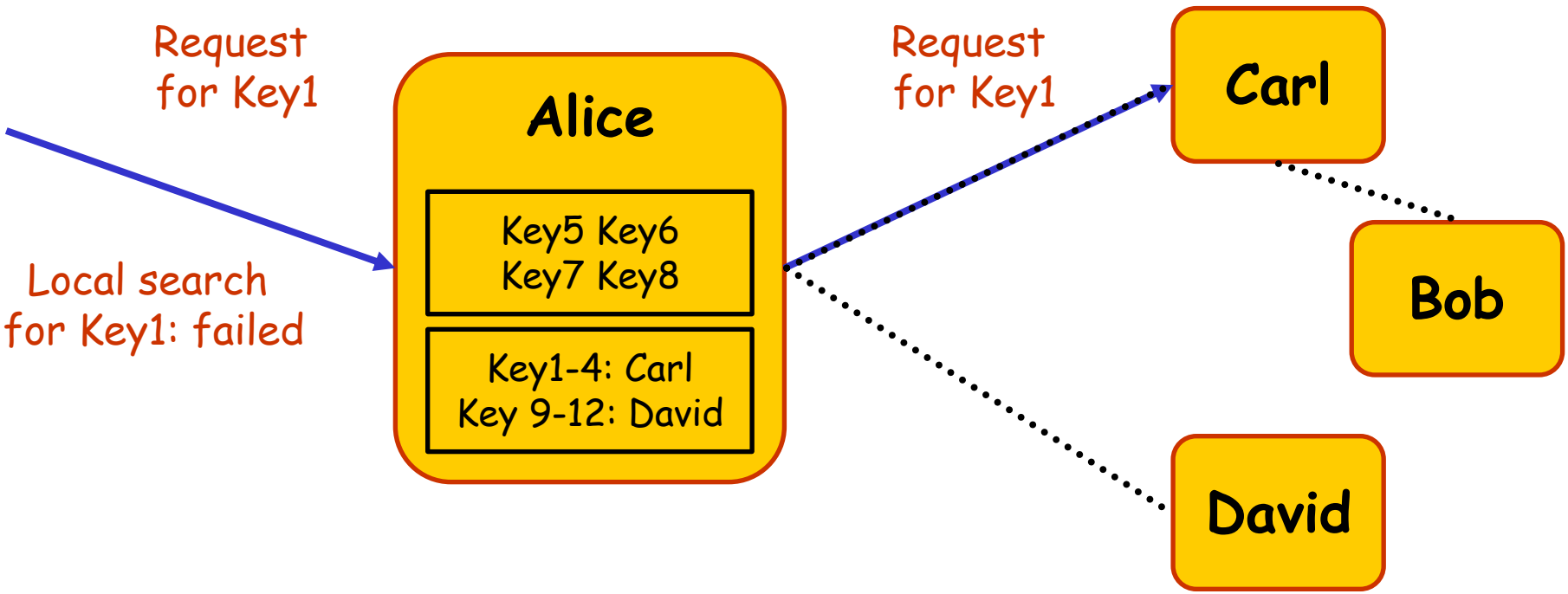
- **Freenet is a P2P network of nodes storing data files**
- **Data files:**
  - are named by location-independent *keys*
  - Keys are obtained by hashing a short text-description of the file
- **Freenet nodes:**
  - maintain a *datastore* and make it available to the network:
    - for reading
    - for writing
  - maintain a *dynamic routing table* containing
    - addresses of other hosts
    - keys they are thought to hold
  - query one another to store and retrieve data files

Freenet Messages

- **All messages contain:**
  - A randomly-generated 64 bit *Transaction ID*
    - Unique “with high probability”
  - A *TTL* field (Hop-to-Live)
  - A *Depth* field (number of hops performed so far)
- **Messages are forwarded from node to node**
  - TTL *decremented* at each hop
    - Message not discarded when TTL reaches 1
    - Randomly forwarded for other steps (for anonymity)
  - Depth *incremented* at each hop
    - Used in reply messages to set the TTL
    - Does not start at 0 (for anonymity)

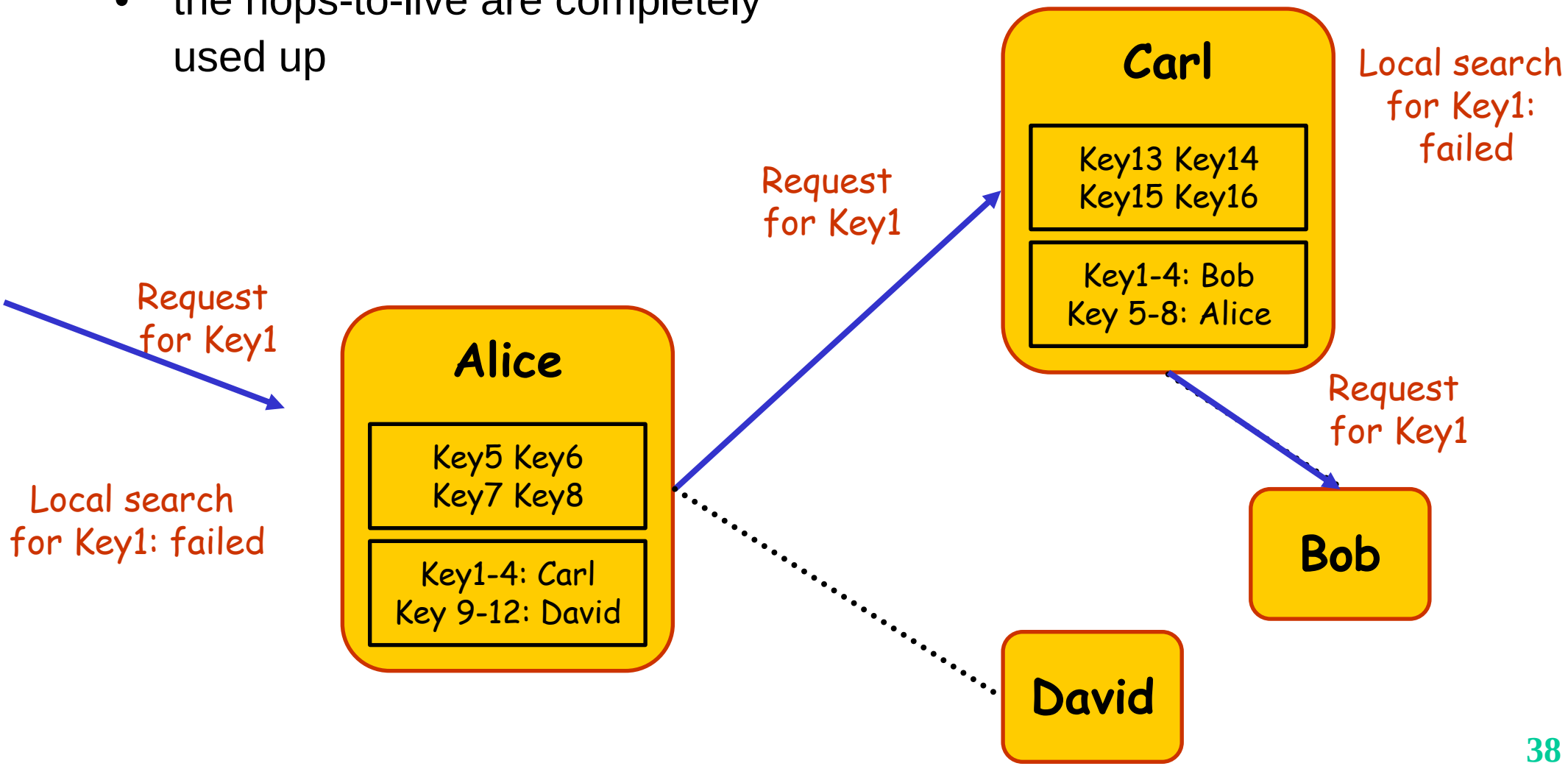
Freenet Algorithm: Request

- **When a node receive a request for a key:**
  - Attempts to retrieve the file locally, if possible
  - Otherwise, forwards the request to another node
    - Which node? Local decision in IP-style
    - Decision depends on the key
    - Depth-first



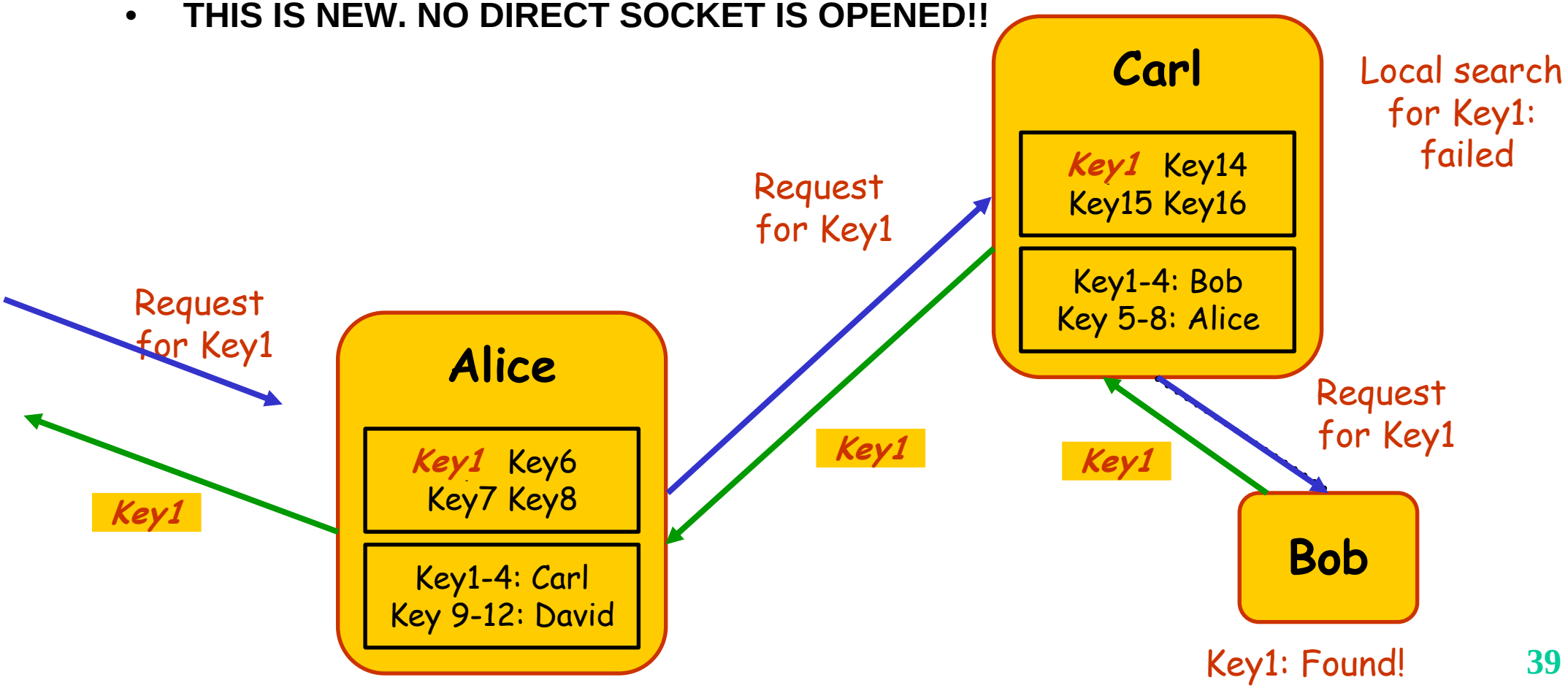
Freenet Algorithm: Request

- **Requests are passed from node to node until**
  - the requested key is found
  - the hops-to-live are completely used up



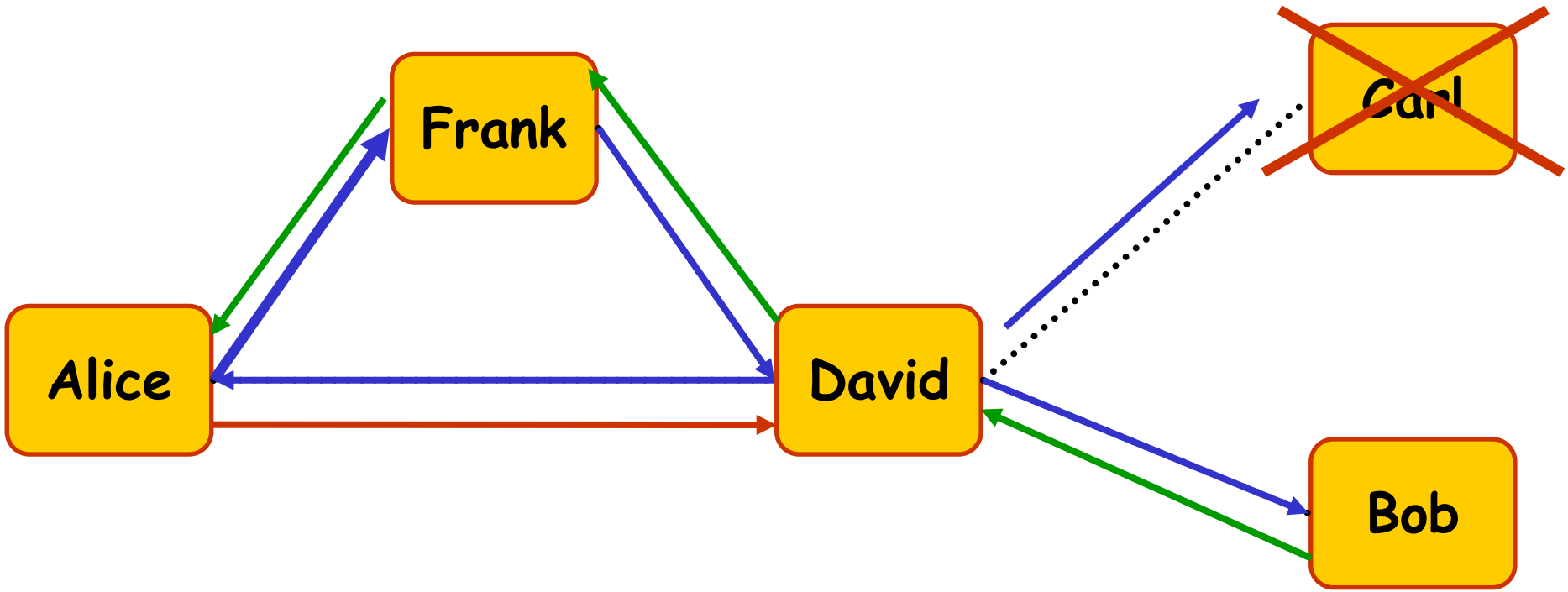
Freenet Algorithm: Request

- **When a request is successful (found in the local datastore)**
  - The data are returned to the requester along the same path of the incoming request
  - The nodes in the return path cache the data in their local datastore
  - **THIS IS NEW. NO DIRECT SOCKET IS OPENED!!**



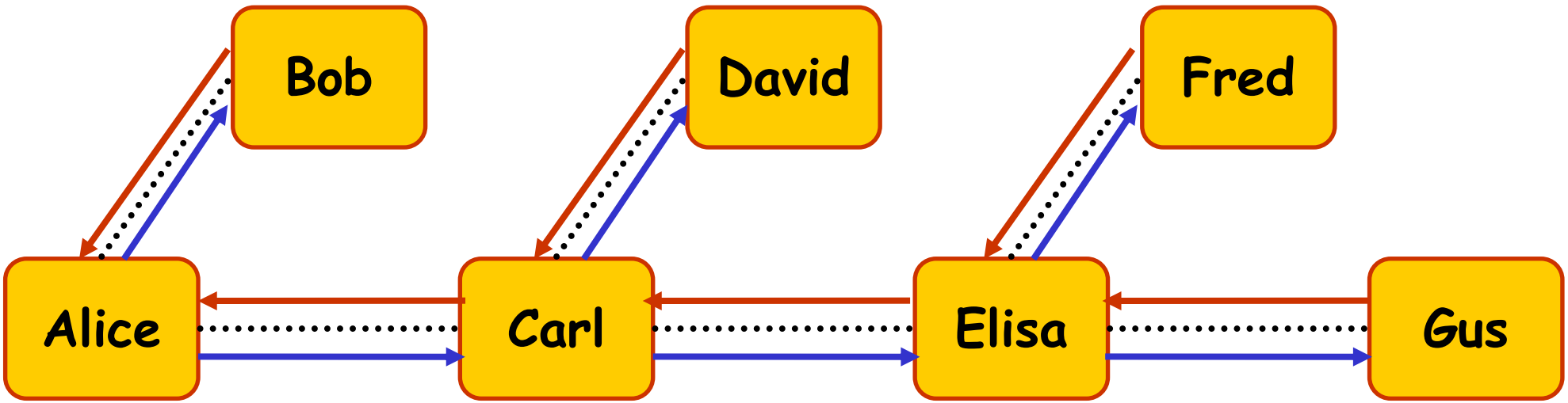
Freenet Algorithm: Request

- **A request cannot be forwarded to the selected node**
  - If the node is down
  - If the node has been already visited by this request
- **In this case, another node among the neighbors is chosen**



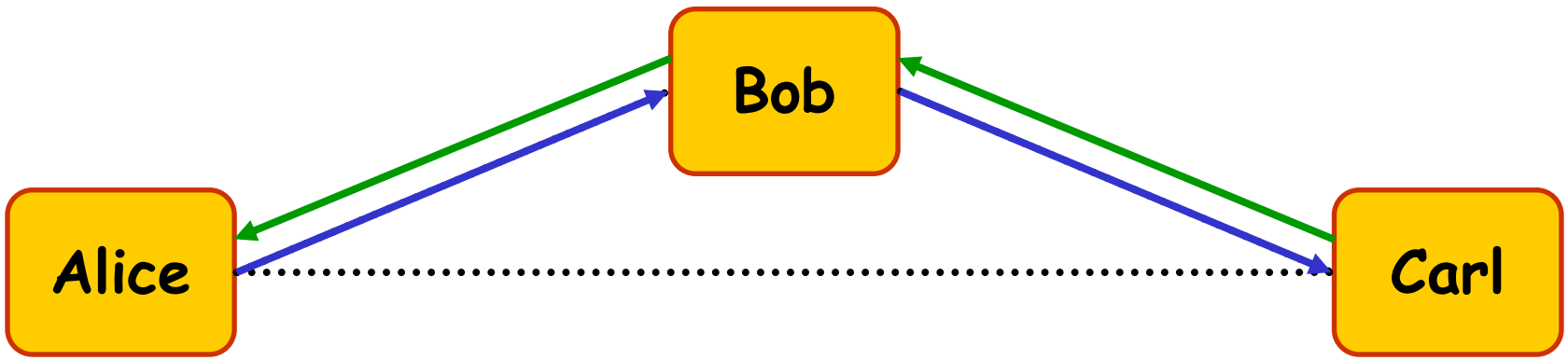
Freenet Algorithm: Request

- **If a node runs out of candidates to try:**
  - it reports failure back to its upstream neighbor
  - this can try other nodes, or report failure back
- **If hops-to-live are exceeded:**
  - a failure is reported back to the original requestor



Freenet Algorithm: Request

- **Network structure actively evolves over time:**
  - Reply messages contain the address of the replying node
  - When receiving a reply message, the replying node is added to the routing table
  - Note: this field may be changed during the return trip (anonymity)



Freenet Algorithm: Data Management

- **Datastores in Freenet nodes:**
  - store file content and key
  - have configurable size
  - use LRU policy
  - when a new file arrive:
    - if enough space is available, store the new file
    - else, delete the LRU file and store the new file
- **Routing tables**
  - use LRU policy in the same way as datastore
  - routing entries will be retained longer since they are smaller

Freenet Algorithm: Data Management

- **Datastores are not caches:**
  - there is no “permanent” copy which is being replicated in a cache
  - when all nodes independently decide to drop a file, the file is no longer available in the network
  - Freenet goal is not persistent storage
- **Stored files are encrypted:**
  - node operators cannot know the content of their datastores
  - the motivations:
    - not for securing files (requestor can read them)
    - node operator can deny any knowledge about its datastore

Keys

- **Files in Freenet are identified by binary file keys:**
  - 160 bit keys
  - Obtained through SHA-1 hash algorithm
- **Three kind of keys:**
  - Keyword-signed key (KSK) (the original)
  - Signed-subspace key (SSK)
  - Content-hash key (CHK)
- **Keys and routing tables:**
  - A node cannot maintain a pair (key, next hop) for each document in the system
  - Lexicographic closeness of keys is used

Freenet Routing

- **Assumption:**
  The quality of routing should improve over time
- **Reasons:**
  - Nodes should specialize in locating sets of similar keys
    - If a node is listed in routing tables for a particular key, it will tend to receive request for keys similar to that key
  - Nodes should specialize in storing clusters of files having similar keys
- **These two effect improve the efficiency of future request:**

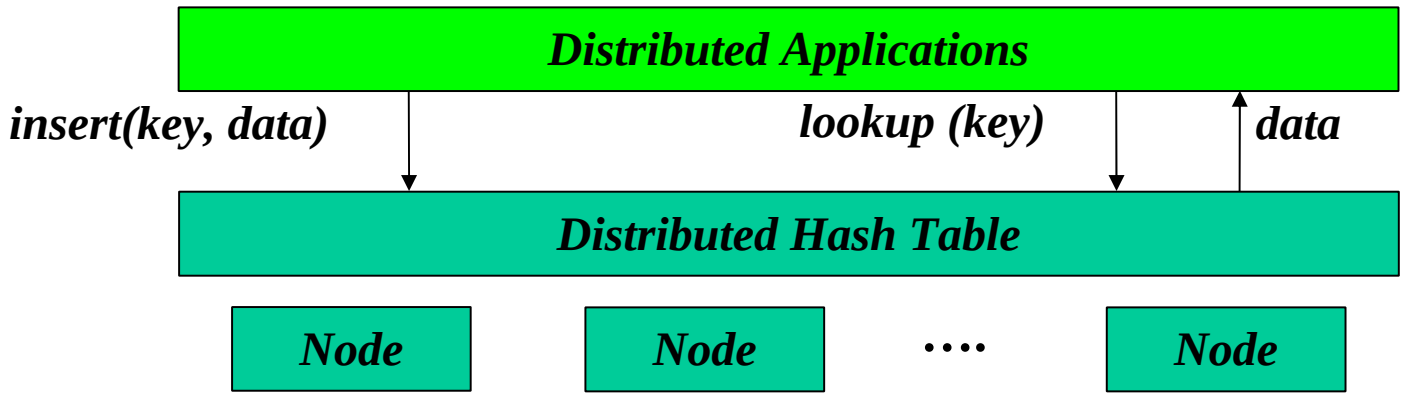
  self-reinforcing cycle
- **Freenet evolves into a small-world network to boost search performance.**

Freenet: Conclusions

- **Search Problem**
  - The keys are Freenet's current weak point
  - Hashing of keys:
    - Human-Rights.doc and HumanRights.doc have completely different hash values
  - Hashing renders Freenet unusable for random searches
    - Need an "out-of-band" communication of keys
- **A solution:**
  - Possibility of using hyperlinks
  - Multiple key-word independent hashing

Distributed Hash Tables

- **Each node handles a portion of the hash space and is responsible for a certain key range**
  - no global knowledge
  - absence of single point of failures
  - greater scalability
  - uniform distribution of resources
  - Examples: CAN, Chord, Pastry, Tapestry

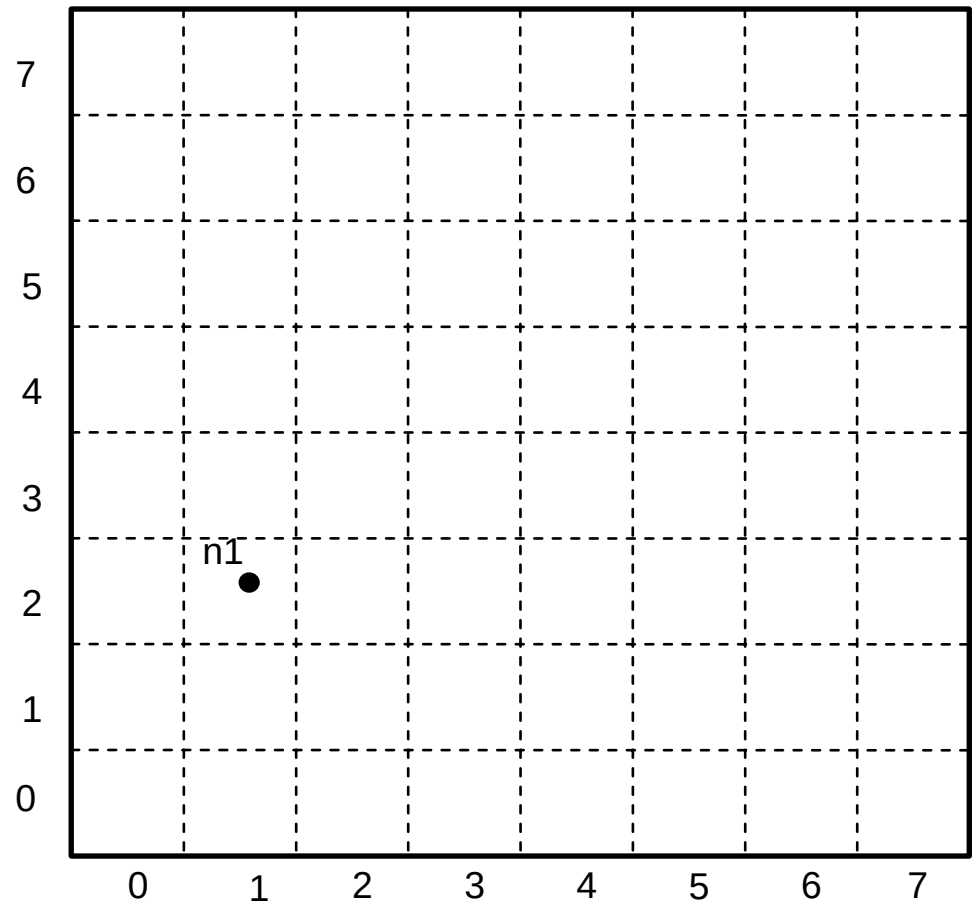


DHT – CAN

- **Associate to each node and item a unique id in an d-dimensional space**
- **Goals**
  - Scales to hundreds of thousands of nodes
  - Handles rapid arrival and failure of nodes
- **Properties**
  - Routing table size O(d)
  - Guarantees that a file is found in at most d*n1/d steps, where n is the total number of nodes

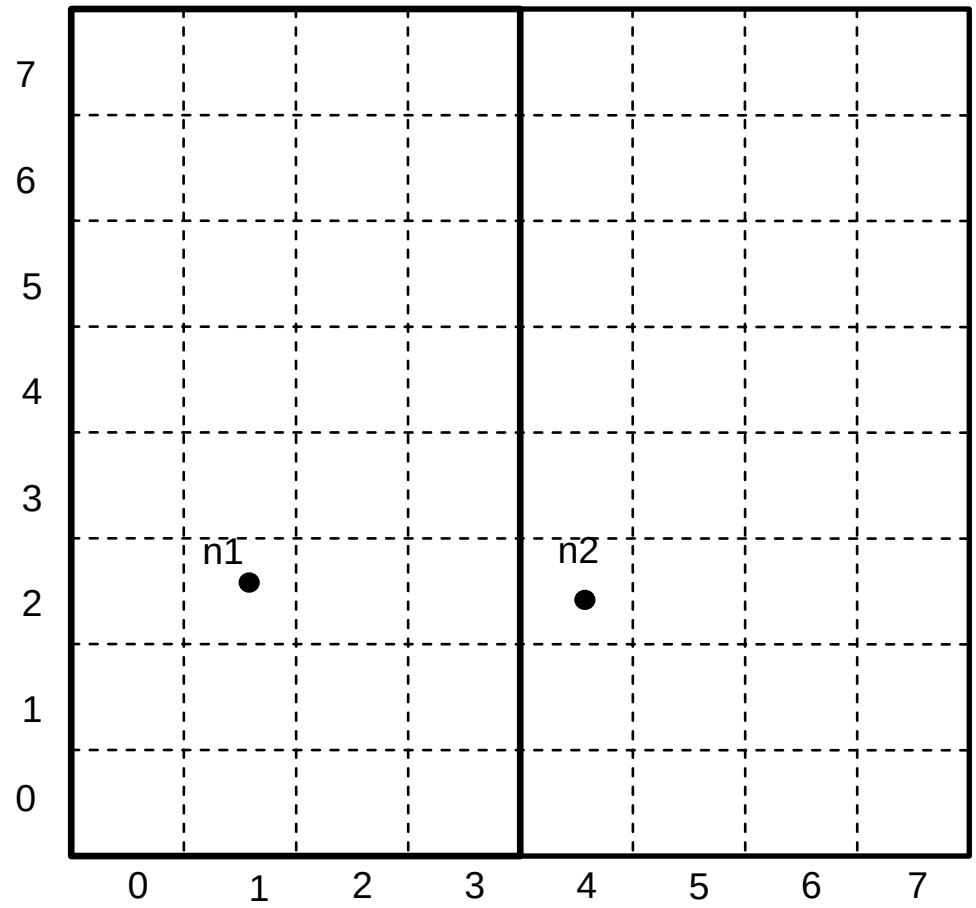
CAN Example: Two Dimensional Space

- **Space divided between nodes**
- **All nodes cover the entire space**
- **Each node covers either a square or a rectangular area of ratios 1:2 or 2:1**
- **Example:**
  - Node n_1:(1, 2) first node that joins, covers the entire space



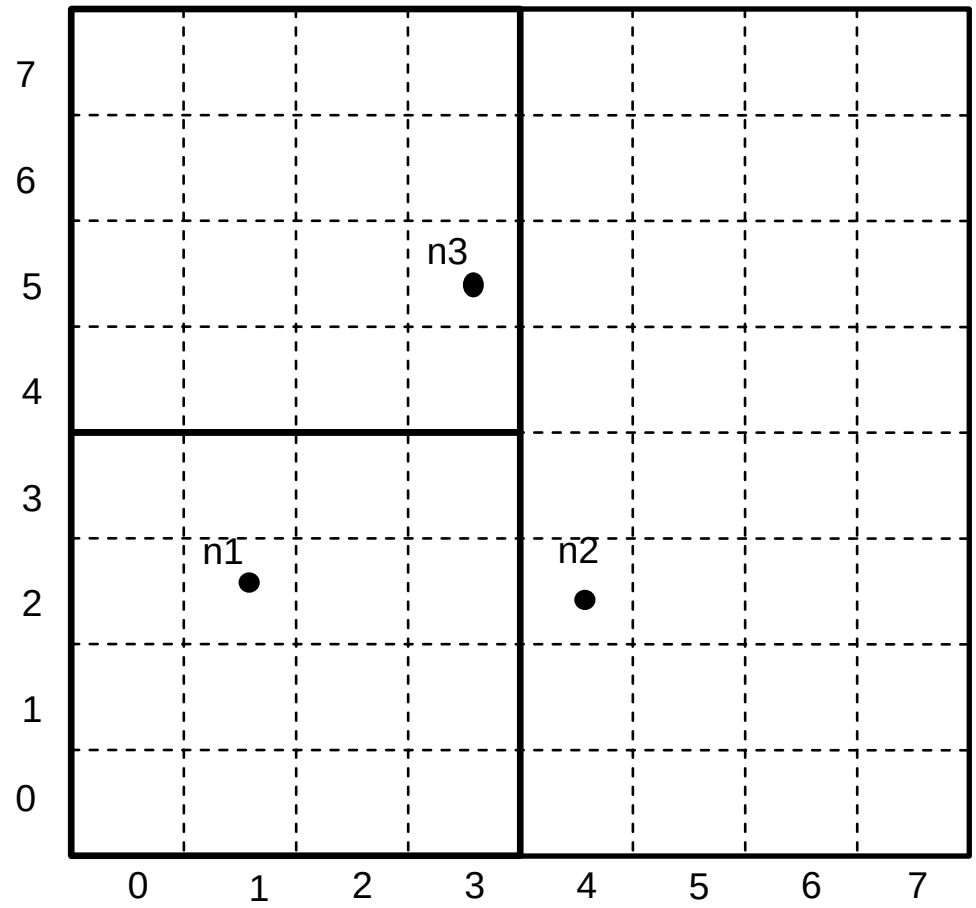
CAN Example: Two Dimensional Space

- **Node n_2:(4, 2) joins**
  - Space is divided between n_1 and n_2



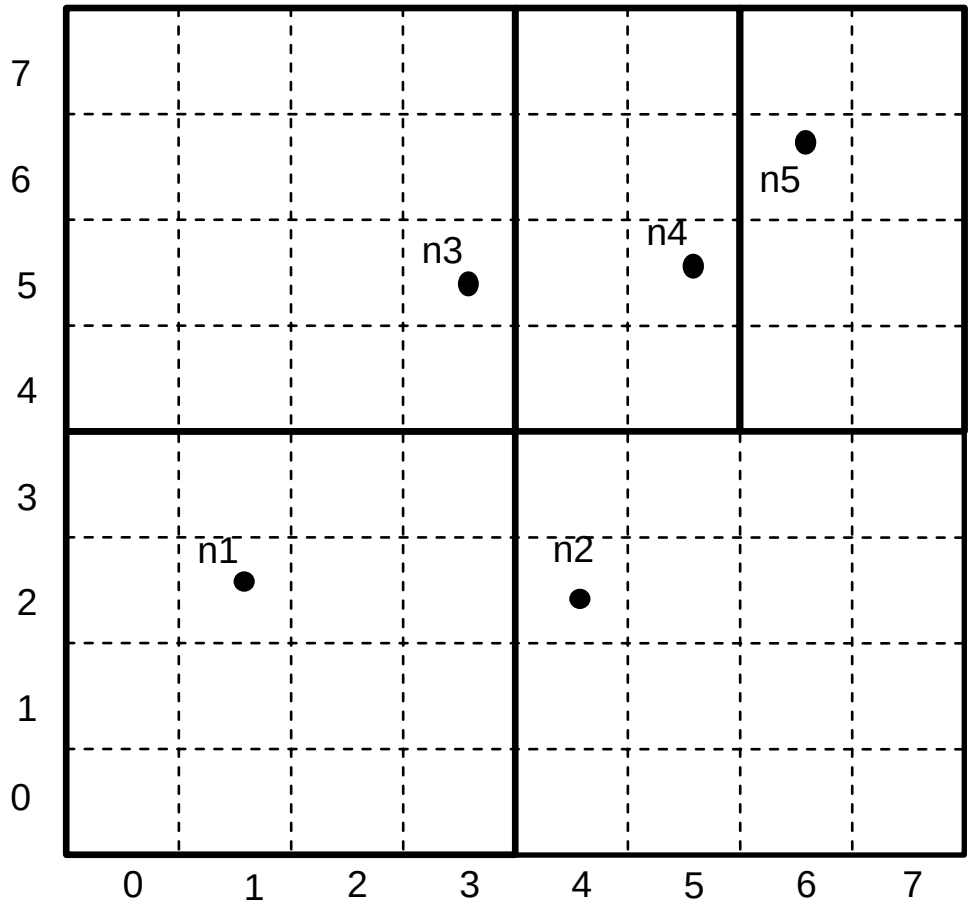
CAN Example: Two Dimensional Space

- **Node n_3:(3,5) joins**
  - Space is divided between n_1 and n_2



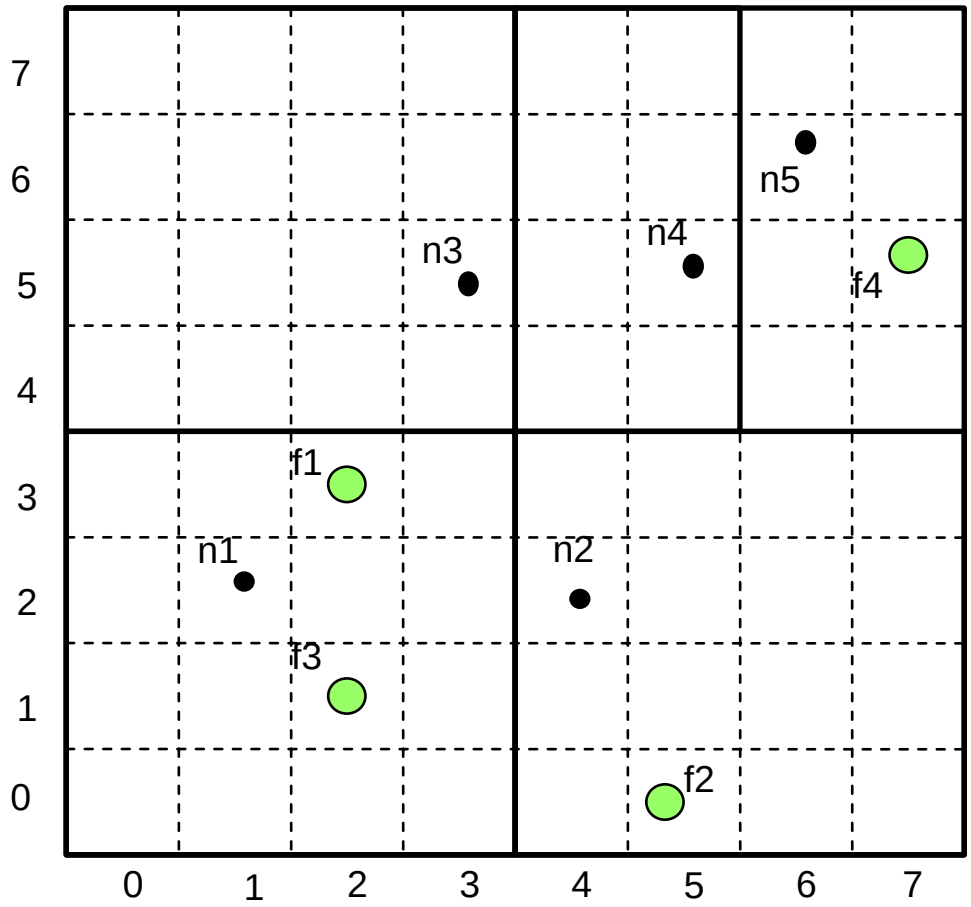
CAN Example: Two Dimensional Space

- **Nodes n_4:(5,5) and n_5:(6,6) join**



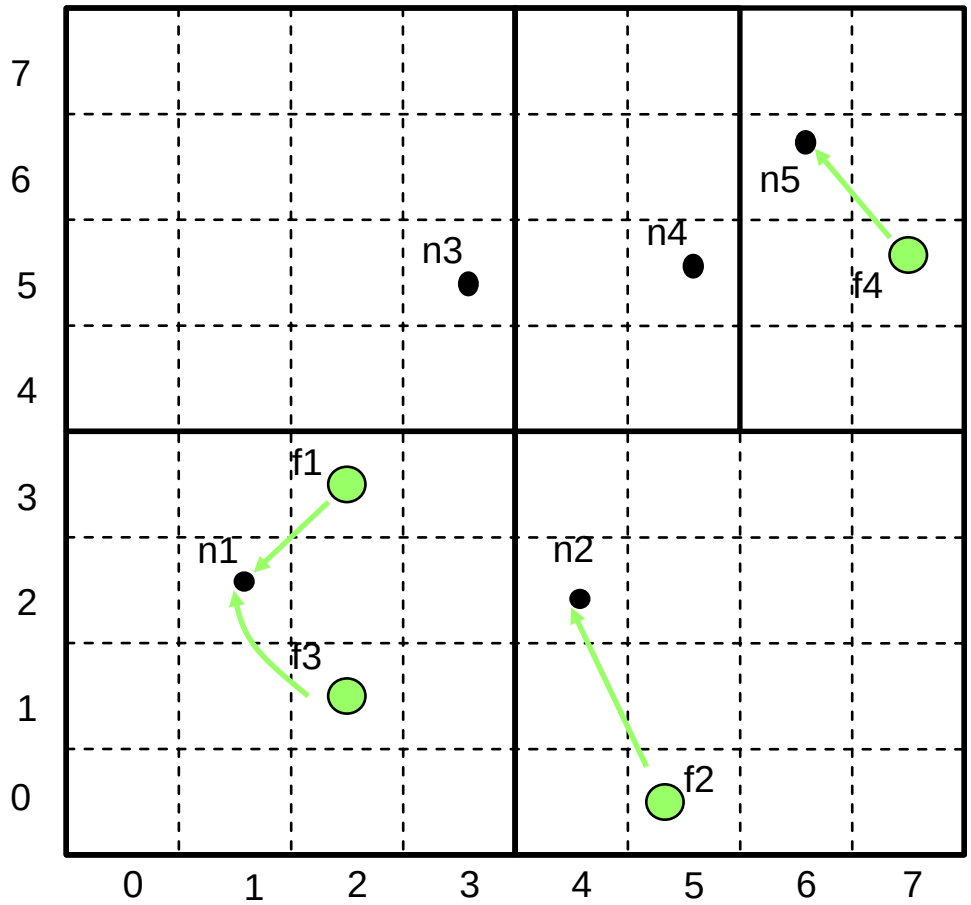
CAN Example: Two Dimensional Space

- *Nodes:*
  - n_1: (1,2)
  - n_2: (4,2)
  - n_3: (3,5)
  - n_4: (5,5)
  - n_5: (6,6)
- *Items:*
  - f_1: (2,3)
  - f_2: (5,1)
  - f_3: (2,1)
  - f_4: (7,5);



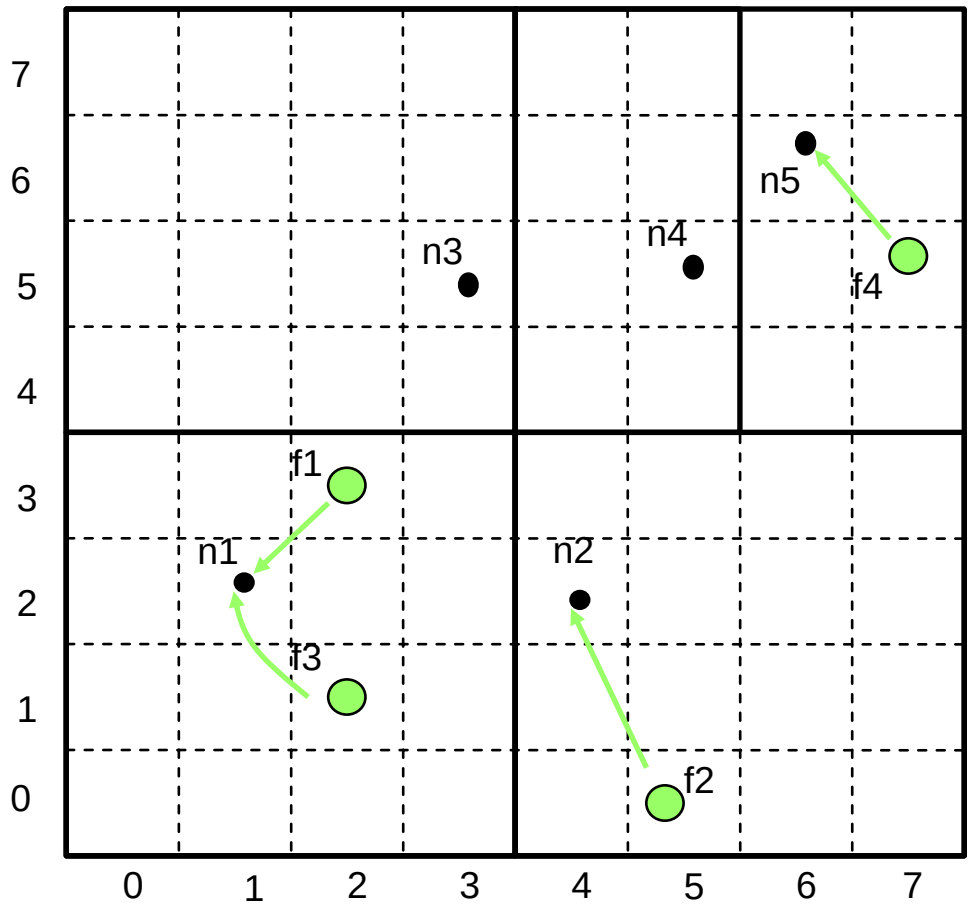
CAN Example: Two Dimensional Space

- **Each item is stored by the node who owns its mapping in the space**



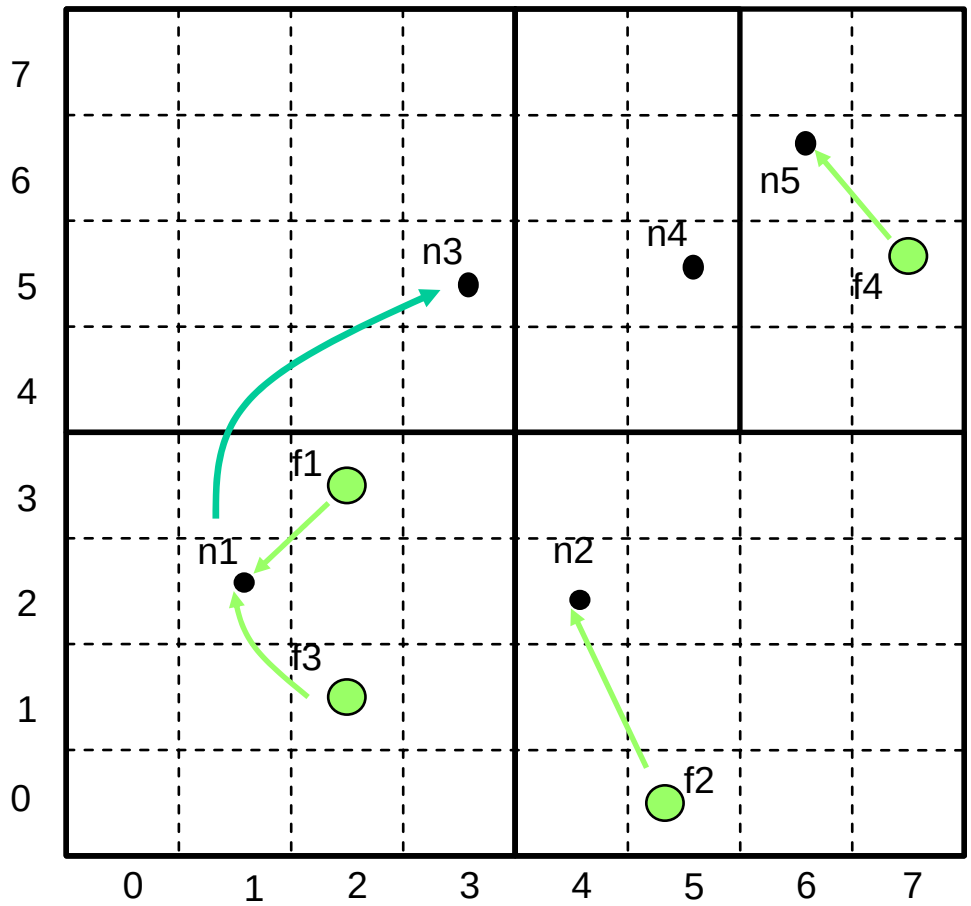
CAN: Query Example

- **Each node knows its neighbors in the d-space**
- **Forward query to the neighbor that is closest to the query id**
- **Example: assume n1 queries f4**
- **Can route around some failures**
  - some failures require local flooding



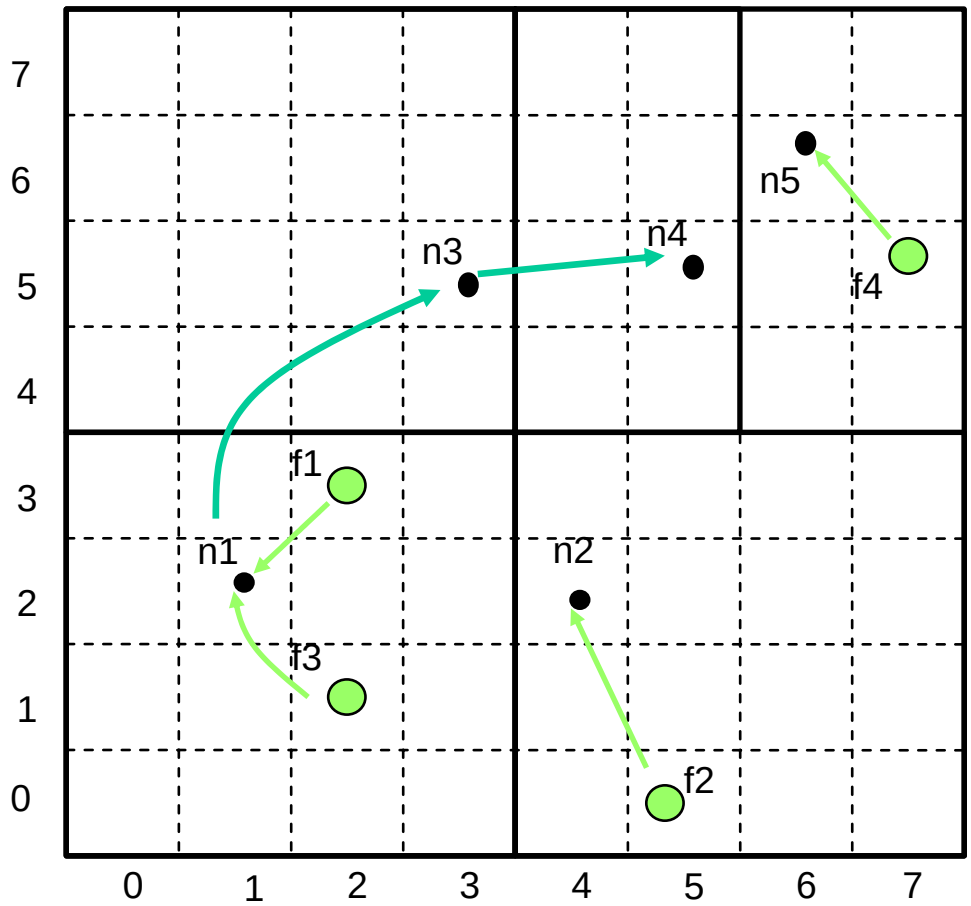
CAN: Query Example

- **Each node knows its neighbors in the d-space**
- **Forward query to the neighbor that is closest to the query id**
- **Example: assume n1 queries f4**
- **Can route around some failures**
  - some failures require local flooding



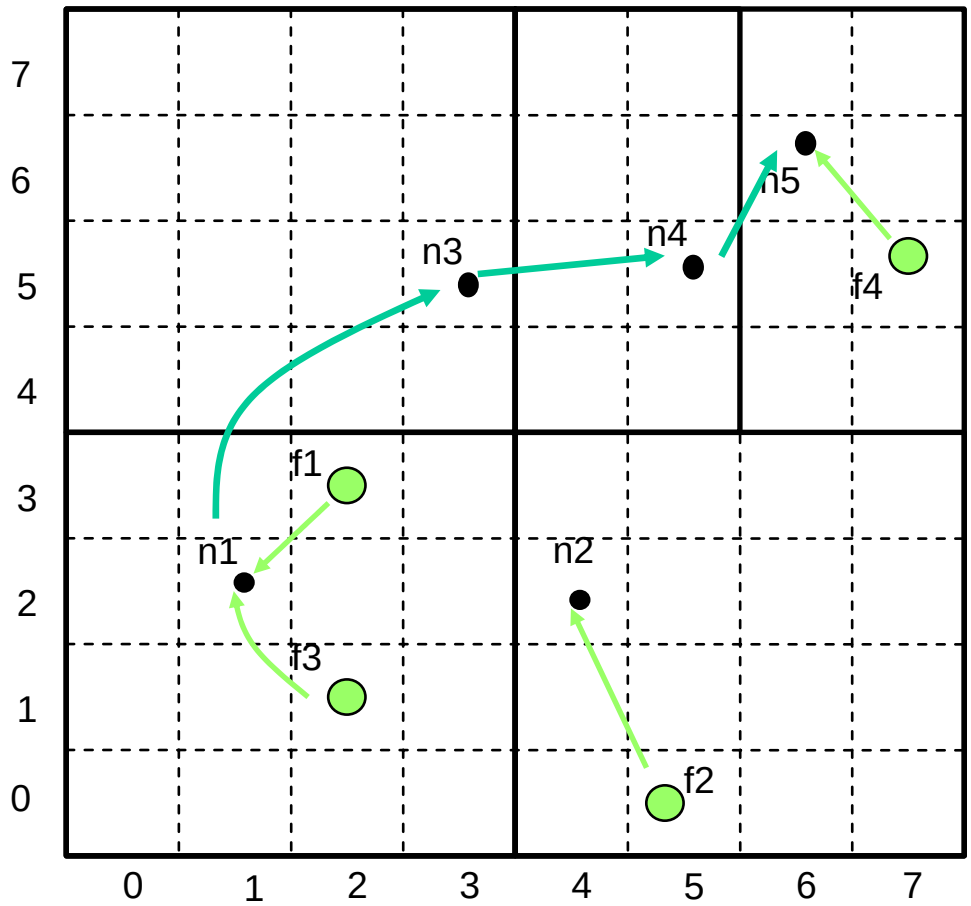
CAN: Query Example

- **Each node knows its neighbors in the d-space**
- **Forward query to the neighbor that is closest to the query id**
- **Example: assume n1 queries f4**
- **Can route around some failures**
  - some failures require local flooding



CAN: Query Example

- **Each node knows its neighbors in the d-space**
- **Forward query to the neighbor that is closest to the query id**
- **Example: assume n1 queries f4**
- **Can route around some failures**
  - some failures require local flooding

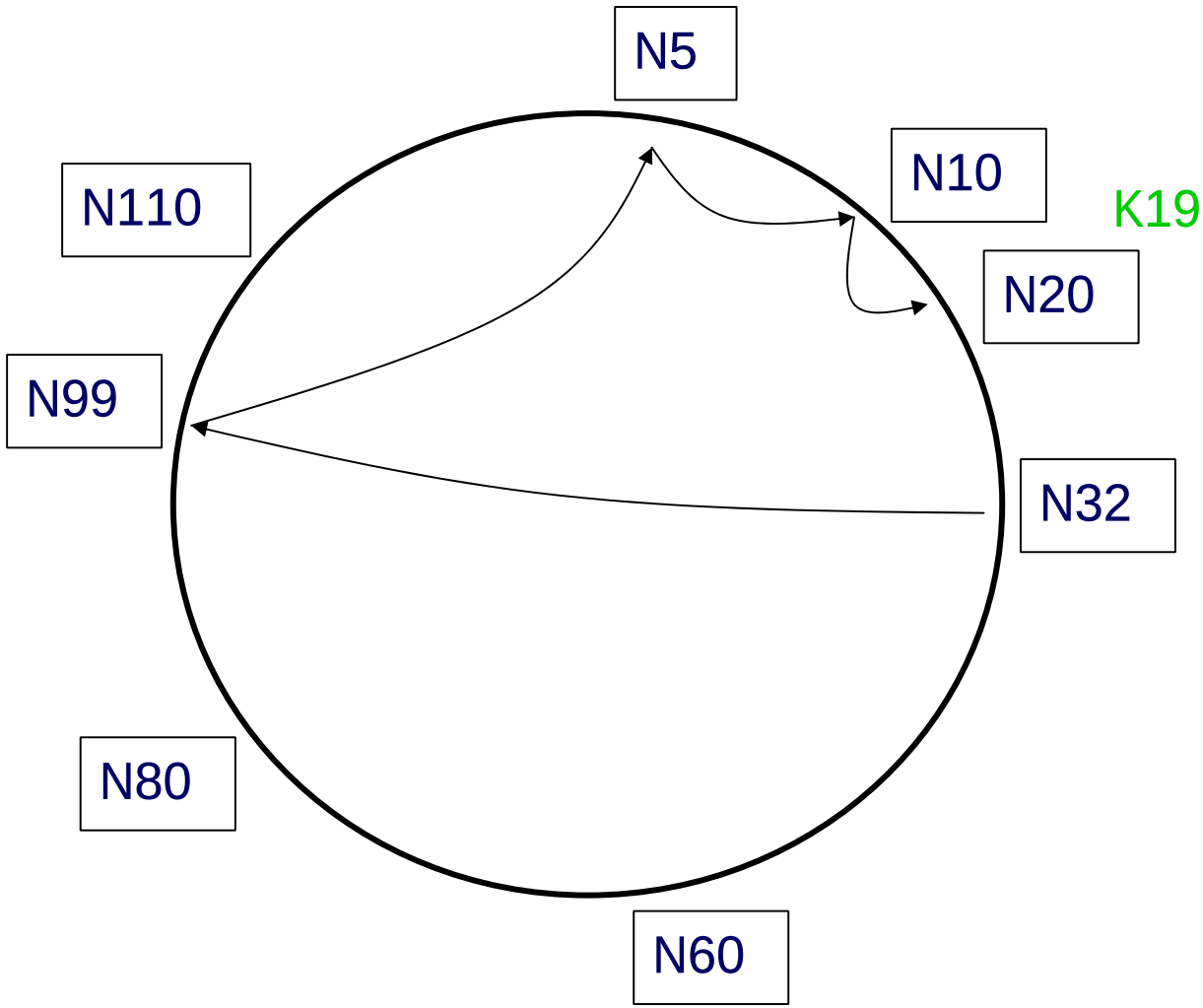


Node Failure Recovery

- **Simple failures**
  - know your neighbor's neighbors
  - when a node fails, one of its neighbors takes over its zone
- **More complex failure modes**
  - simultaneous failure of multiple adjacent nodes
  - scoped flooding to discover neighbors
  - hopefully, a rare event

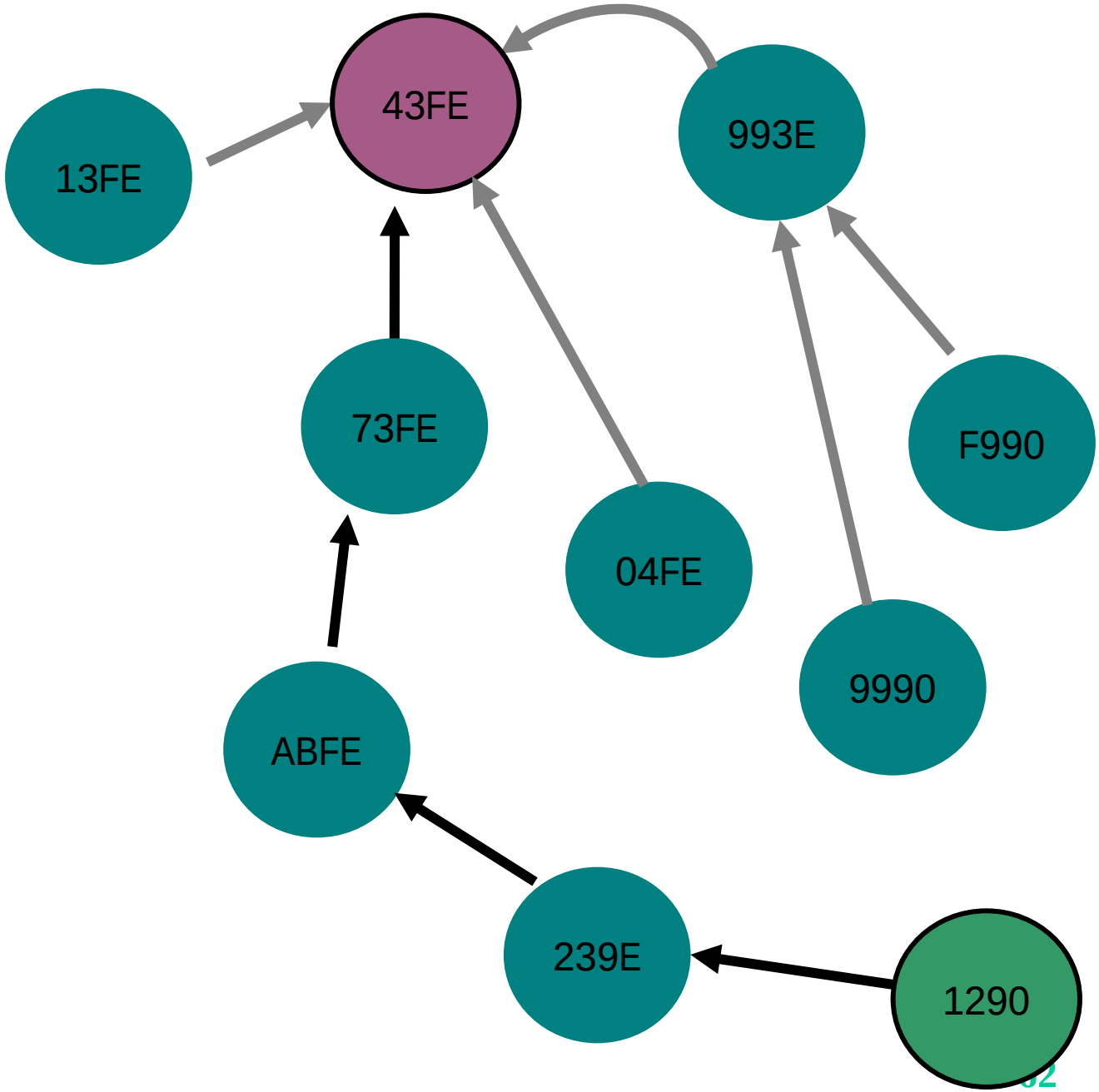
Document Routing – Chord

- **MIT project**
- **Uni-dimensional ID space**
- **Keep track of log N nodes**
- **Search through log N nodes to find desired key**



Doc Routing – Tapestry/Pastry

- **Global mesh**
- **Suffix-based routing**
- **Uses underlying network distance in constructing mesh**



Comparing Guarantees

	Model	Search	State
Chord	Uni-dimensional	log N	log N
CAN	Multi-dimensional	$dN^{1/d}$	2d
Tapestry	Global Mesh	$\log_b N$	$b \log_b N$
Pastry	Neighbor map	$\log_b N$	$b \log_b N + b$

DHT: Conclusions

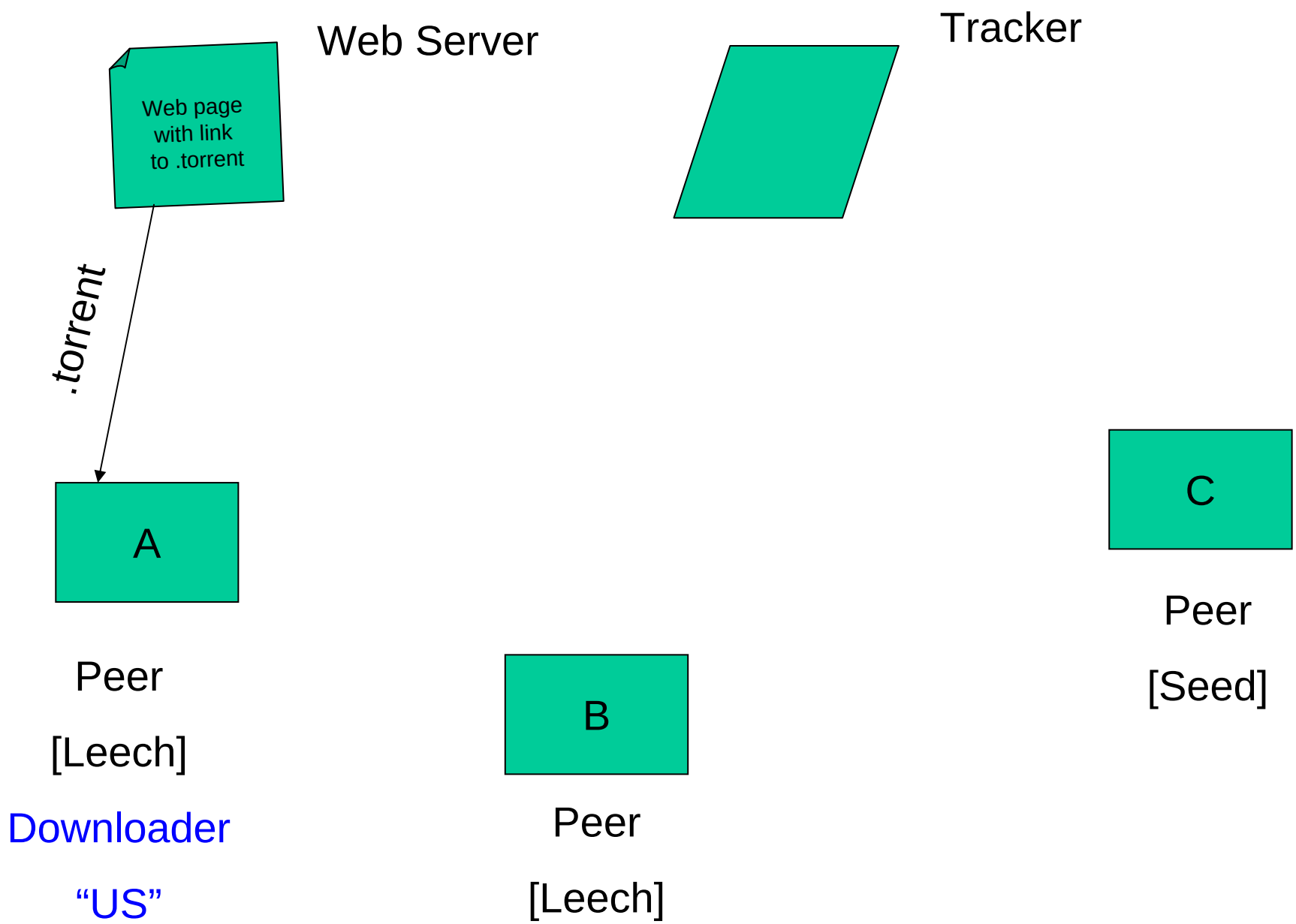
- **Problems**
  - Hard to handle highly dynamic environments
  - Most of these mechanisms do not consider peer characteristics
  - Key Hashing destroy semantics
- **Recent Trends**
  - Multi-key hashing
  - Physical network coupling
- **Possible applications**
  - Publish-subscribe (Scribe)
  - Distributed File Systems (OceanStore)
  - ...

Bit Torrent

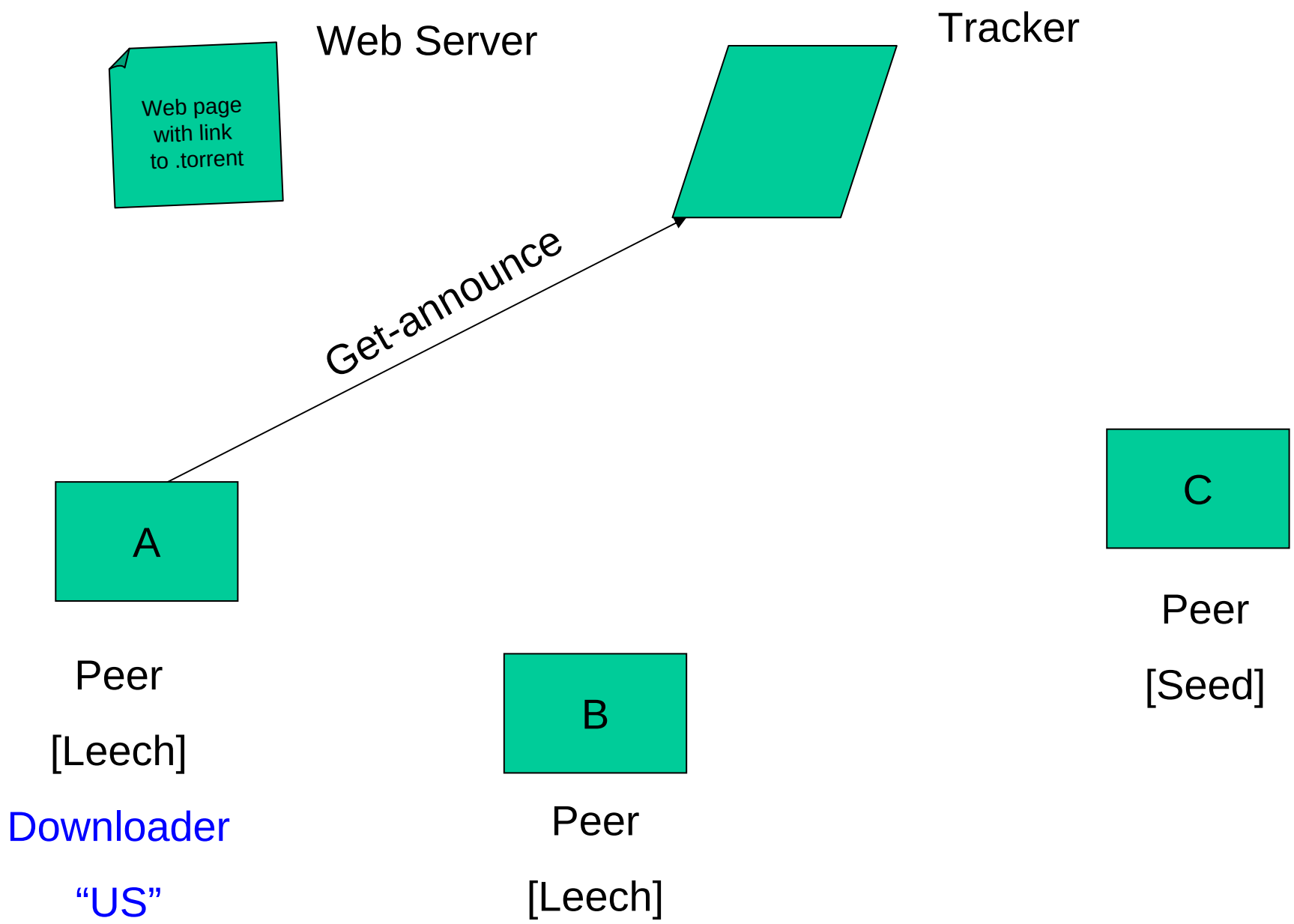
- **Based on Tit-for-tat (to avoid free-riding)**
  - Incentive - Uploading while downloading
- **Pieces of files**
  - To increase dowload speed
  - Heaviliy exploit replicas



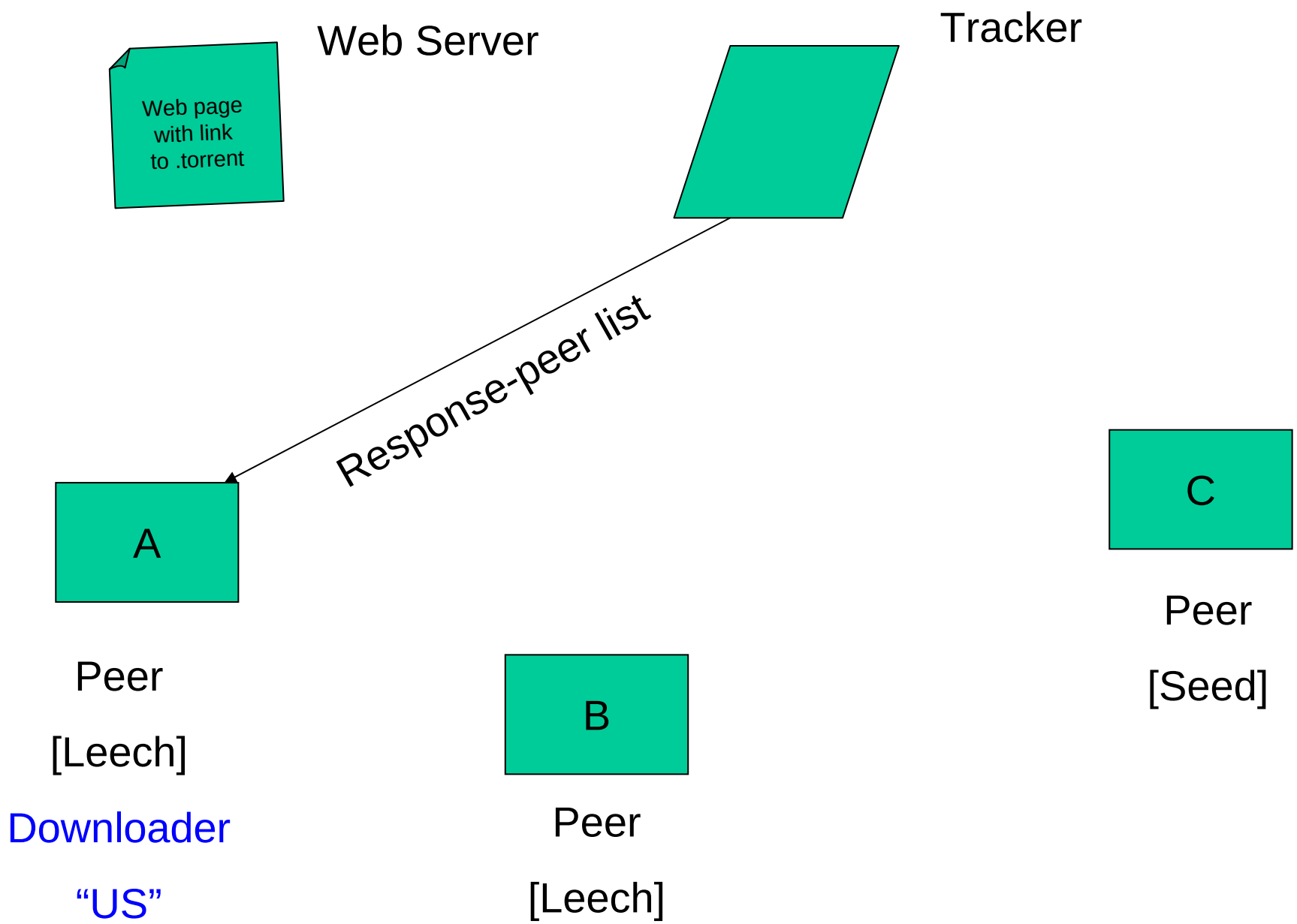
Overall Architecture



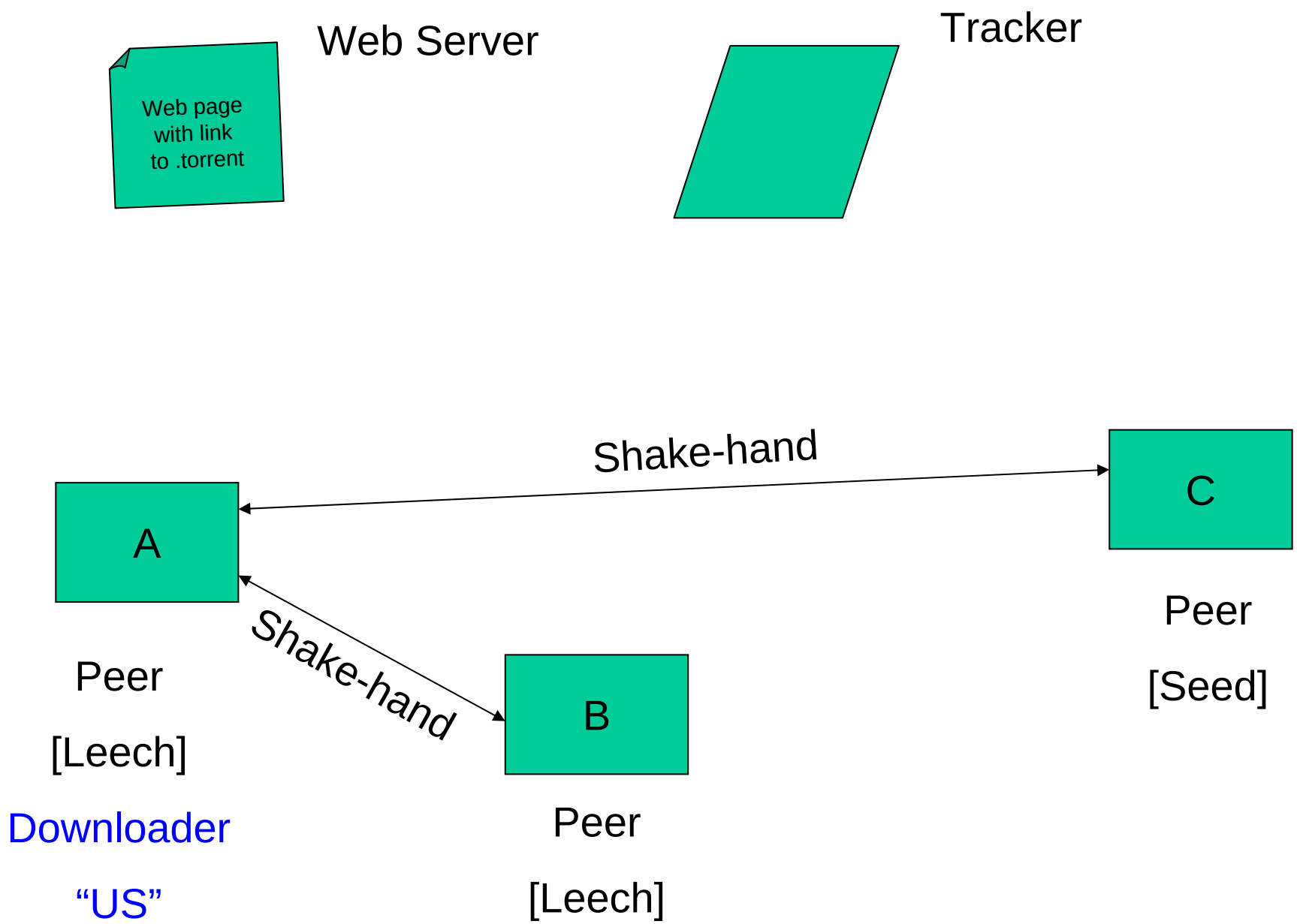
Overall Architecture



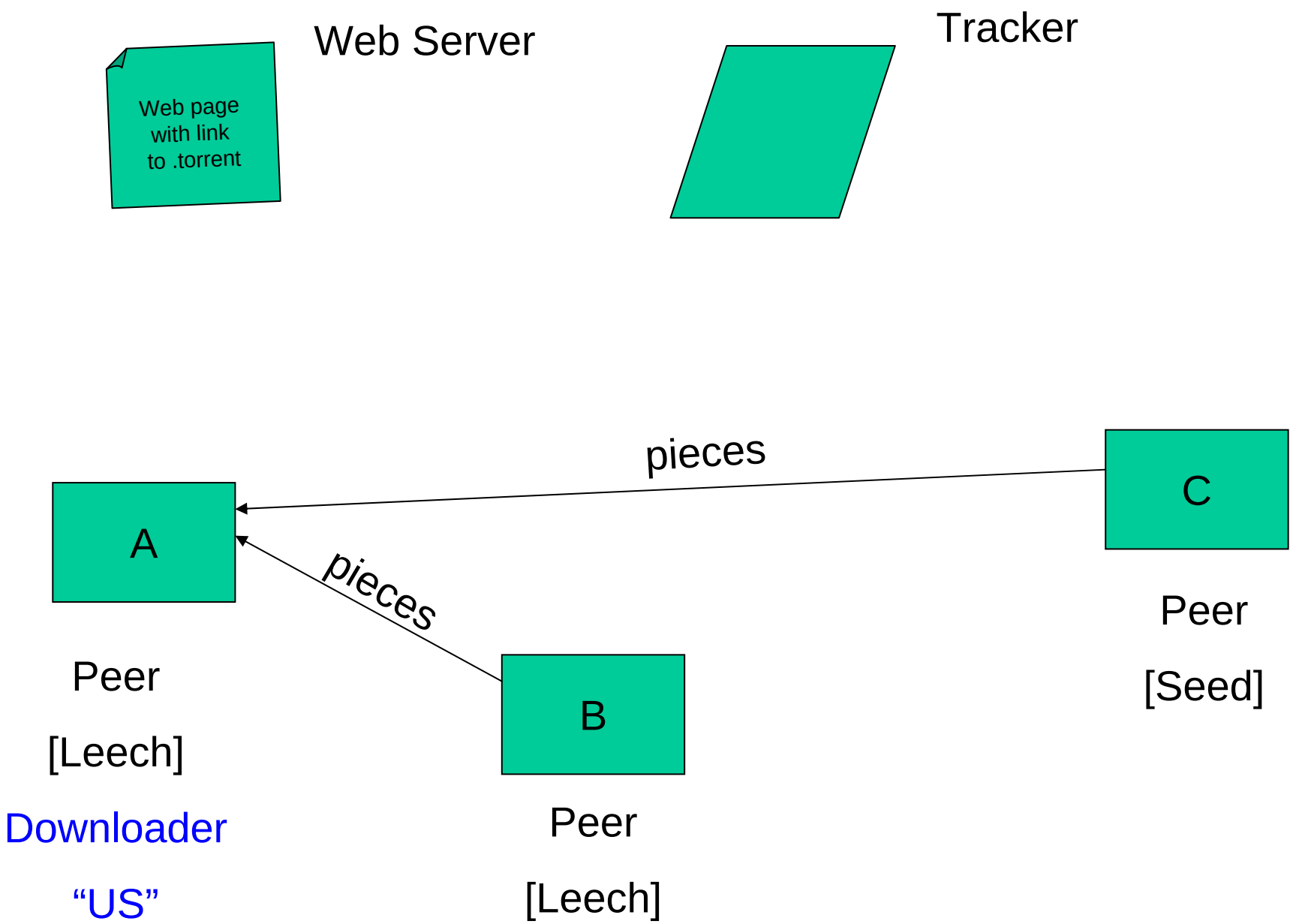
Overall Architecture



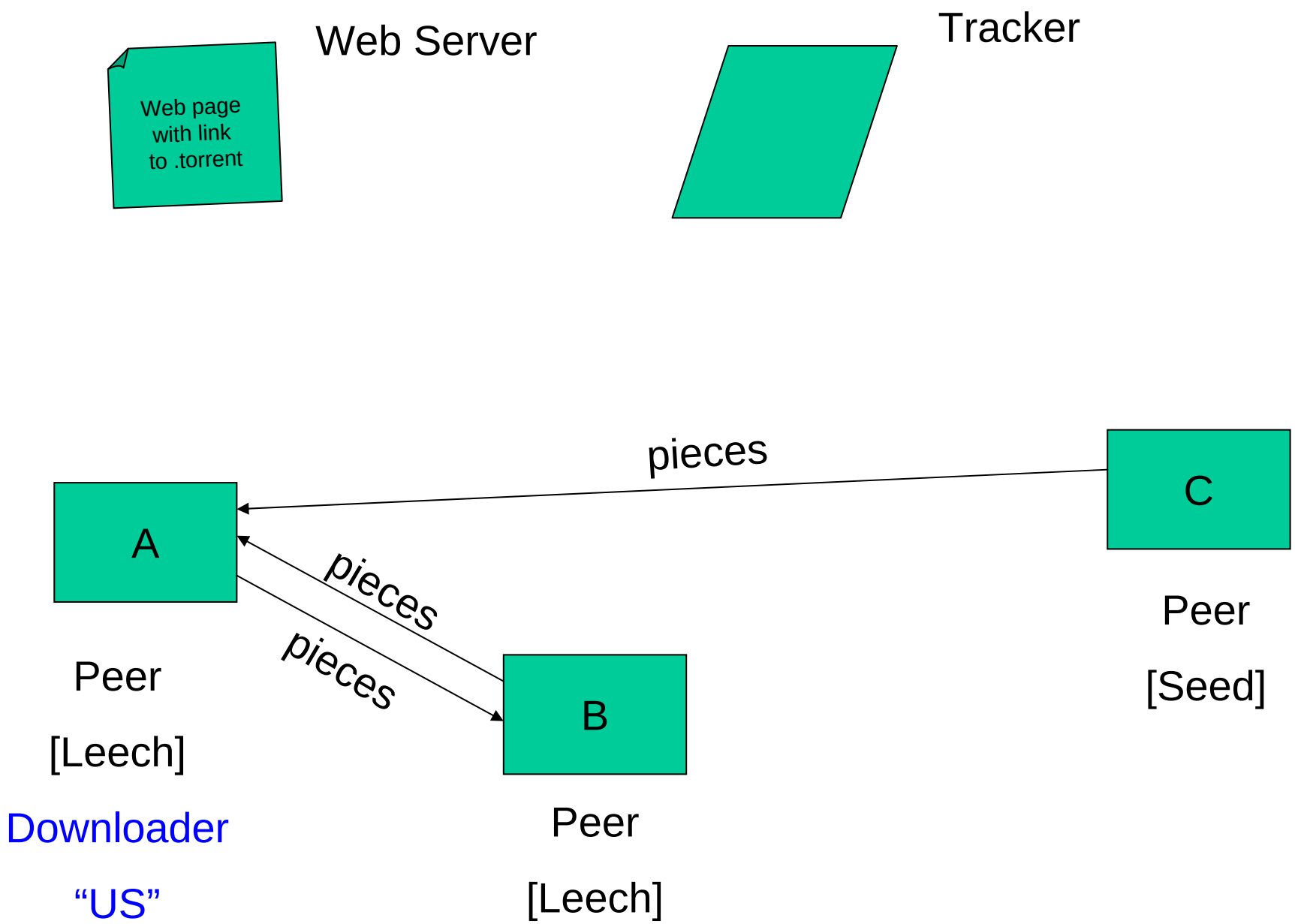
Overall Architecture



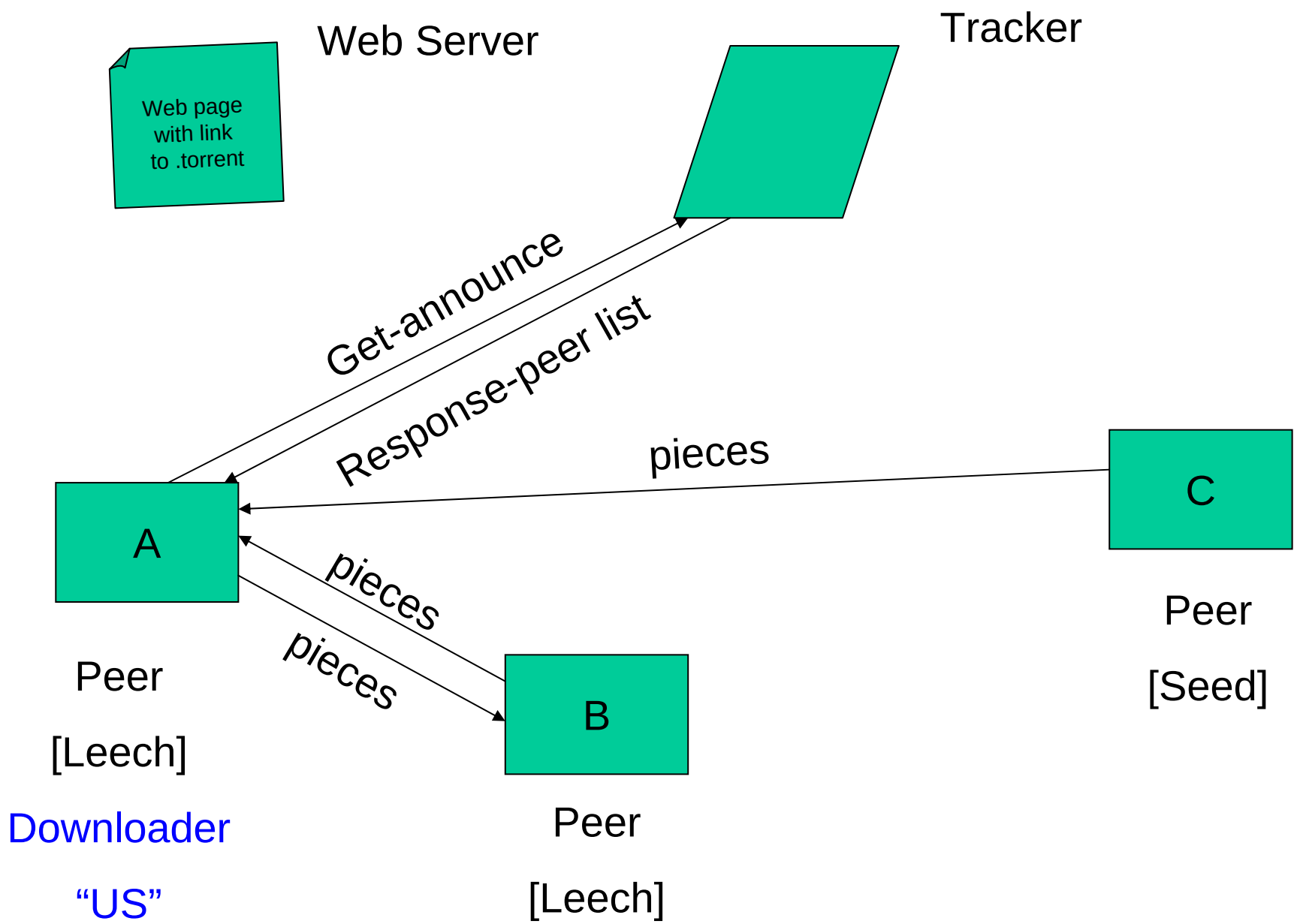
Overall Architecture



Overall Architecture



Overall Architecture



BitTorrent Messages and Components

- **Peer – Peer messages**
  - TCP Sockets
- **Peer – Tracker messages**
  - HTTP Request/Response
- **Torrent File (.torrent)**
  - URL of the tracker
  - Pieces <hash1,hash2,….hash_n>
  - Piece length
  - Name
  - Length
  - Files
- **Tracker**
  - Peer cache
    - IP, port, peer id
  - State information
    - Completed
    - Downloading
  - Returns random list
- **Pieces Selection**
  - rarest first
  - Random first piece
  - redundant requests in endgame

Peer Selection

- **Built-in incentive mechanism*:***
- ***Choking Algorithm***
  - Choking is a temporal refusal to upload
  - Choking evaluation is performed every 10 seconds.
  - Each peer unchokes a fixed number of peers (default = 4)
  - The decision on which peers to un/choke is based solely on download rate, which is evaluated on a rolling, 20-second average - ***tit for tat***
- ***Optimistic Unchoking***
  - a BitTorrent peer has a single '*optimistic unchoke*' which is uploaded regardless of the current download rate from it. This peer rotates every 30s
  - Reason: To discover currently unused connections that are better than the ones being used

P2P Initiatives

- **P2P projects share common problems**
  - Multitude of P2P projects appeared recently
  - Common problems: security, scalability, reliability, routing
- **Unfortunately:**
  - A theoretical and technical framework capable of supporting P2P application development is still missing
  - Traditional techniques, given the substantial increase in the *complexity* of these systems, are not suitable
- **This lack has been recently recognized by the IT community**

P2P Initiatives: JXTA

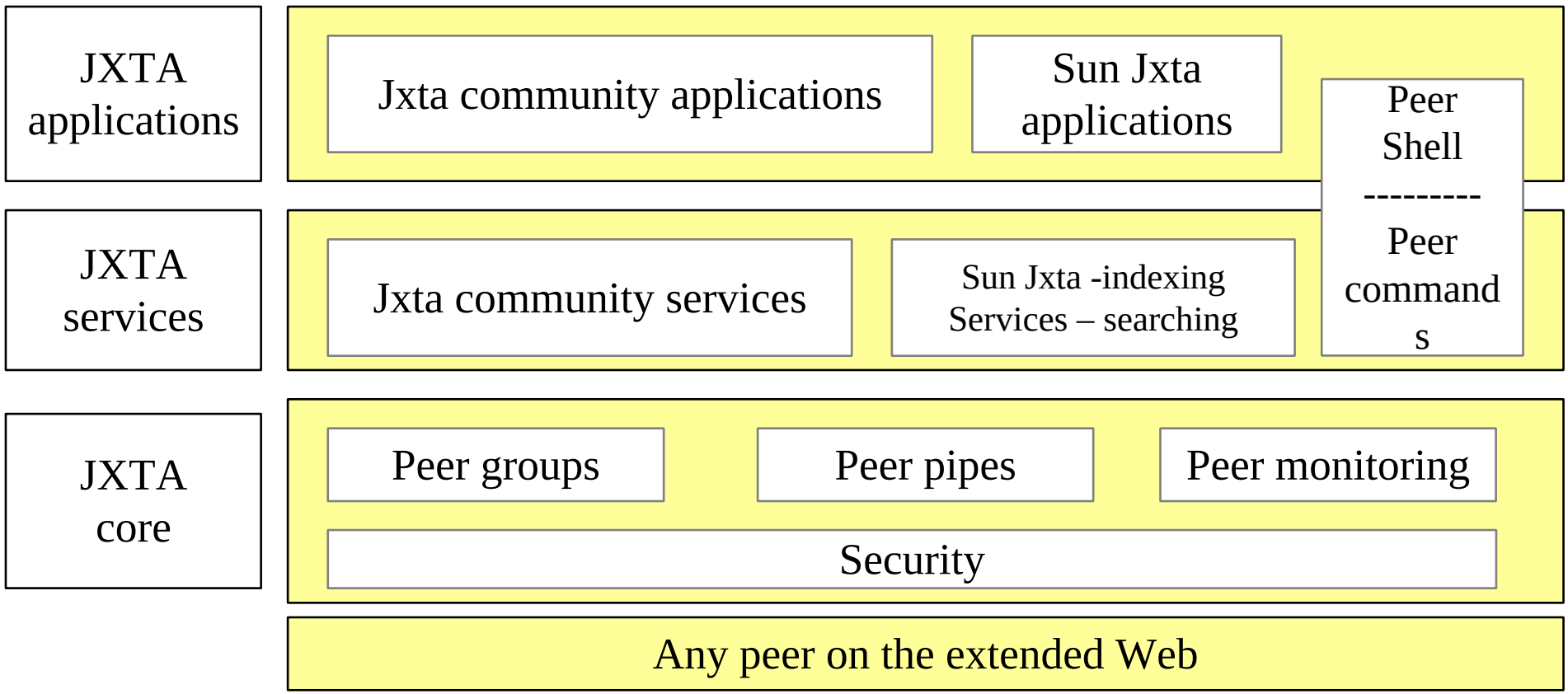
- **The acronym JXTA comes from "*JuXTApose*"; according to Bill Joy (chief scientist at Sun):**

  "[Juxtapose] means putting things next to each other, which is really what peer-to-peer is about"

- **JXTA is aimed at providing minimal mechanisms by which people can further innovate:**
  - being able to pipe from one peer to another
  - being able to group a set of of peers together, and create groups of groups
  - doing monitoring and metering
  - a security layer

Jxta P2P Software Architecture

- **Jxta Three Layer Cake**



Conclusions and discussion

- **Is it all hype?**
- **Should P2P be a research area?**
- **Do P2P applications/systems have common research questions?**
- **What are the “killer apps” for P2P systems?**
- **Can be used in other scenarios (Ubicomp)?**